

天津大学

《软件工程综合实践》结课报告



学 院 智能与计算学部

专 业 软件工程

年 级 2023级

小 组 第04小组

学号	姓名
3023208045	高灿
3023244158	曾意
3023244168	贾思韵
3023244157	戴茗静
3023244362	杨晓越

2025 年 10 月 7 日

目 录

第一章 软件需求规格说明书	1
1.1 引言	1
1.1.1 简介	1
1.1.2 背景	1
1.1.3 定义、缩略语	3
1.2 需求分析	3
1.2.1 目标	3
1.2.2 涉众分析	3
1.3 业务分析	4
1.3.1 业务分析类图	4
1.3.3 业务流程分析 (UML活动图)	7
1.4 功能性需求	9
1.4.1 总用例	9
1.4.2 用户的用例	9
1.4.3 顾客的用例	11
1.4.4 商家的用例	18
1.4.5 管理员的用例	24
1.4.6 其他功能性需求	28
1.5 非功能性需求	28
1.5.1 系统架构	28
1.5.2 安全性	29
第二章 项目设计文档	30
2.1 数据库设计	30
2.1.1 DB 一览表	30
2.1.2 表结构	30
2.2 后端接口设计	37
2.2.1 商铺管理部分接口设计	37
2.2.2 点赞收藏部分	39

2.2.2 用户模块接口设计	40
2.2.3 地址模块接口设计	42
2.2.4 权限申请管理接口设计	42
2.2.5 管理员模块接口设计	43
2.2.6 消息与通知模块接口设计	44
2.2.7 文件上传模块接口设计	44
2.2.8 商品模块接口设计	44
2.2.9 购物车模块接口设计	46
2.2.10 订单模块接口设计	47
2.3 前端设计	48
2.3.1 普通用户	48
2.3.2 商家用户	59
2.3.3 管理员用户	62
2.4 安全设计	65
2.4.1 密码安全策略	65
2.4.2 身份认证机制	66
2.5 细节设计	68
2.5.1 公用返回组件设计	68
2.5.2 显示设计	68
2.5.3 历史订单修改逻辑设计	68
2.5.4 手机号验证设计	68
2.5.5 部分逻辑优化设计	68
第三章 人员分工	69
第四章 项目开发过程	70
4.1 第一阶段 (9.8-9.13)	70
4.1.1 第一次组会记录 (9.8)	70
4.1.2 第二次组会记录 (9.10)	71
4.2 第二阶段 (9.14-9.18)	71
4.2.1 第三次组会 (9.15)	71
4.3 第三阶段 (9.19-9.20)	72

4.3.1 第四次组会记录 (9.19)	72
4.4 第四阶段 (9.21-9.23)	72
4.4.1 第五次组会记录 (9.21)	72
4.4.2 第六次组会记录 (9.23)	73
4.5 第五阶段 (9.24-9.25)	73
4.2.1 第一轮全面测试	73
4.2.2 第七次会议 (9.25)	73
4.6 第六阶段 (9.26-9.27)	74
4.6.1 完成最终展示 (9.26)	74
4.6.2 第八次会议 (9.26)	74
4.6.3 文档、报告、总结等后续工作	74
第五章 项目测试文档	75
5.1 测试概括	75
5.1.1 项目背景	75
5.1.2 编写目的	75
5.2 前端测试	75
5.2.1 测试目标	75
5.2.2 测试范围	75
5.2.3 测试方法	76
5.2.4 测试执行	76
5.2.5 测试内容	76
5.2.6 测试结果	78
5.3 后端测试	78
5.3.1 测试内容	78
5.4 前后端联合测试	81
5.4.1 测试范围与核心场景	81
5.4.2 关键问题排查与验证方法	82
第六章 部署文档	83
6.1 开发环境	83
6.2 运维需求	85

6.3 部署的步骤和流程	85
6.4 部署展示	85
第七章 项目特色说明文档	87
7.1 文档目的与术语说明	87
7.1.1 文档目的	87
7.1.2 术语定义	87
7.2 系统功能详述	87
7.2.1 顾客端功能	87
7.2.2 商家端功能	90
7.2.3 管理端功能	91
7.3 项目特色与创新点	92
第八章 问题与解决&结果反思	93
第九章 实践总结与展望	97
9.1 实践总结	97
9.2 未来展望	97

第一章 软件需求规格说明书

1.1 引言

1.1.1 简介

本文档面向“饿了么”APP项目的客户决策者、项目经理、设计与开发团队、测试人员及相关参与方，旨在系统阐述产品完整需求。文档综合文字、UML图、ER图、活动图、类图、用例图等多种形式，明确规定了APP的功能组成、性能要求、用户界面、运行环境、外部接口及用户交互响应机制，既用于客户确认产品需求，也为开发提供清晰依据，同时支撑后续测试验收和系统迭代。

1.1.2 背景

(1) 项目背景

在没有“饿了么”系统前，用户订餐需通过电话或第三方平台完成，订单管理混乱、易出错，商户菜单更新不及时，支付方式单一，用户体验较差。日常运营中出现以下问题：

- ① 订单依赖人工记录和处理，错误频出且效率低下
- ② 商户无法自主管理商品和订单，沟通成本高
- ③ 用户无法便捷地查询历史订单或管理个人偏好
- ④ 缺少系统化的购物车和地址管理，用户下单流程繁琐

在某次促销活动中，因订单量激增，人工处理出现大量错误，导致用户投诉激增、商户结算混乱，该事件促使管理层决定开发“饿了么”系统，以实现订单自动化处理、提升用户体验、增强商户自主管理能力。

(2) 项目核心价值

- ① 提升运营效率：通过自动化订单处理和商户自助管理，大幅减少人工干预
- ② 改善用户体验：提供完整的购物车、账号管理和订单跟踪功能，提升下单便捷性
- ③ 增强数据利用：系统化收集用户行为数据，为后续优化提供支持
- ④ 支持业务扩展：提供稳定、可扩展的系统基础，支持未来功能迭代和业务增长

(3) 开发路线

基于公司的技术基础和偏好，本项目后续开发将沿用之前采用的前后端分离的技术路线，主要技术栈如下：

后端：SpringBoot、MyBatis、WebSocket

前端：VUE 3

数据库：MySQL

(4)项目现状

有利条件：

- ①已有部分订单处理和用户管理模块的基础代码
- ②技术栈成熟，前后端框架选型明确，社区资源丰富
- ③可基于常见电商和外卖模式设计功能，业务逻辑有大量参考案例

不利条件：

- ①初始代码存在设计缺陷和功能漏洞，需大量重构
- ②支付等外部接口依赖第三方，集成存在不确定性
- ③时间紧迫，仅有三周时间，完整实现所有功能并保证稳定性挑战较大

(5)目前项目工作内容

1) 架构与全局优化

- 1. 对现有API接口进行规范化改造，全面采用RESTful风格设计，提升接口一致性和可维护性
- 2. 重构登录与鉴权模块，引入Token机制，加强对用户密码的加密存储保护
- 3. 优化系统并发处理能力，在关键业务模块中引入多线程技术
- 4. 将图片等静态资源迁移至对象存储服务（OSS），提升资源访问效率与可靠性

2) 核心功能扩展与完善

- 1. 增强用户个人信息管理功能，支持用户修改昵称、密码、头像及绑定手机号
- 2. 新增收藏功能，支持用户对商品进行收藏、取消及查看收藏列表
- 3. 实现点赞与评论功能，增强用户互动性和内容丰富度

3) 完善搜索功能，支持历史搜索记录显示与快速检索

- 5. 扩展商家端商品管理能力，支持商品下架与信息修改

4) 订单与业务流程优化

- 1. 实施订单超时自动处理机制，订单创建后特定时间未支付将自动关闭
- 2. 强化商家端订单管理，完整显示所有订单及其状态变化
- 3. 细化订单状态流转，新增“已接单”、“已取消”、“已完成”等状态
- 4. 允许用户在一定条件下自主取消订单
- 5. 将价格计算逻辑由前端迁移至后端，确保数据一致性与安全性

5) 智能化升级

探索AI技术在推荐、搜索等场景的应用，提升系统智能化水平

6) 接口设计约束

前后端分离的设计方案，前后端使用符合 RESTful 风格的接口设计。

1.1.3 定义、缩略语

表 1-1 定义与缩略语

术语	解释
购物车	用户在浏览商家菜单时选择的多个商品和商品的临时集合，用户可以在下单之前将这些选定的商品放入购物车中
商家入驻	商家提交资质、签订协议，完成开店流程，包括设置店铺信息、上传营业执照等
订单状态流转	订单经历“待支付”、“待接单”、“已接单”、“已完成”、“已取消”的状态变化
促销与优惠券	平台或商家发起满减、折扣、优惠券等活动，刺激用户下单。

1.2 需求分析

1.2.1 目标

饿了么致力于打造一个高效便捷的在线外卖服务平台，为用户与商家提供优质的数字餐饮解决方案。用户可通过平台实时查看周边餐厅、快速完成选餐下单，并享受高效的配送服务，从而获得简单流畅的订餐体验；同时，餐厅也可借助平台提升订单处理能力与运营效率。平台核心功能包括：

1. 提供用户及商家的账户注册功能
2. 支持用户与商家灵活修改个人账户信息
3. 覆盖用户从浏览商品、管理购物车、调整配送地址到完成支付的全流程点餐服务
4. 实现商家对商品的上架、下架及信息维护等操作
5. 设立管理员权限，支持对商家信息的审核与管理。

1.2.2 涉众分析

注：本系统只涉及第 1、2、3 类涉众, 没有第 4 类涉众，即不与实际支付接口对接，不提供针对配送员的业务流程。

表 1-2 涉众分析

序号	涉众	待解决的问题/对系统的期望
1	顾客	注册账号 管理账号和个人信息 点餐, 管理购物车、送货地址, 下单支付 申请成为商家/切换到商家端
2	商家	切换到顾客端 申请开店 管理店铺 管理商家信息 上架、下架、管理待售商品 接单/拒单/完成订单等订单管理操作
3	管理员	管理用户数据 审核成为商家用户申请 审核商家用户开店申请
4	配送员 (骑手)	获取订单信息 获取商家以及用户地点进行取餐, 送餐

1.3 业务分析

该项目管理的业务主要是以下六个部分: 订单业务、用户基本信息业务、商铺管理业务、商品管理业务、点赞收藏评分业务、用户管理业务。

1.3.1 业务分析类图

(1) 订单

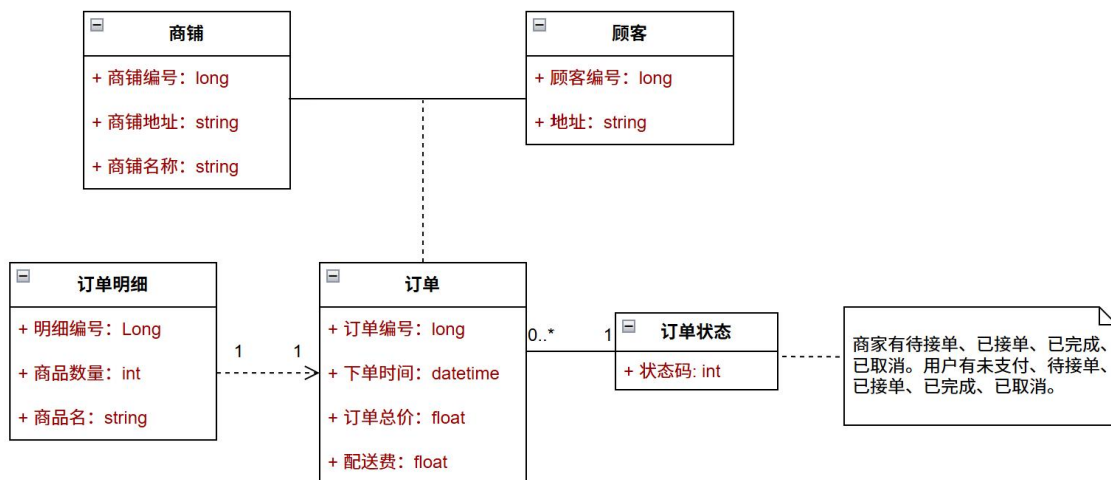


图 1-1 订单类图

(2) 用户基本信息

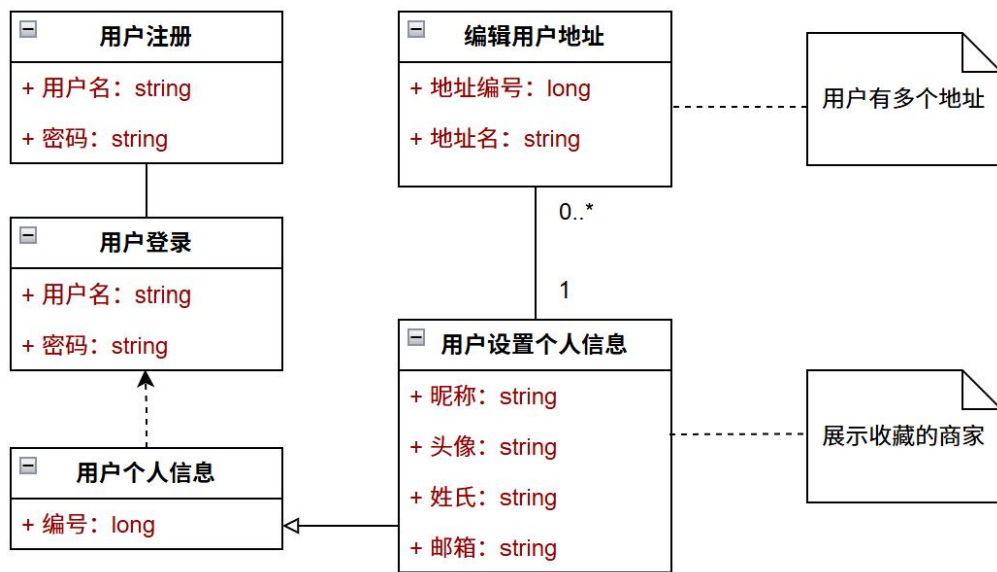


图 1-2 用户类图

(4) 商品管理

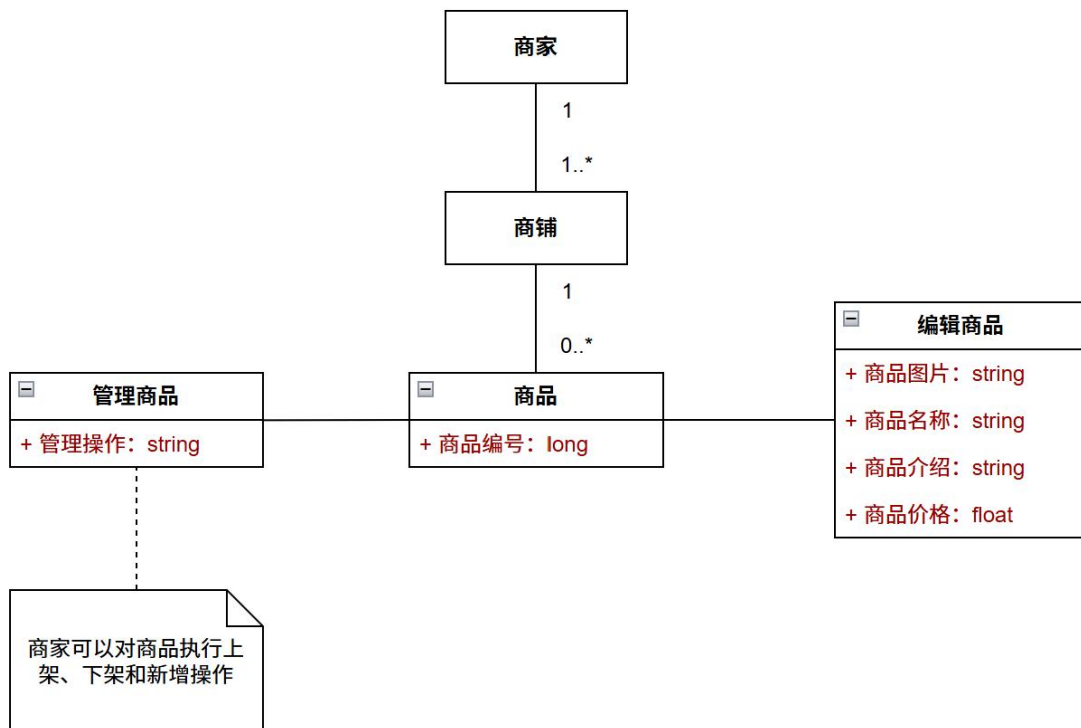


图 1-3 商品管理类图

(4) 商铺管理

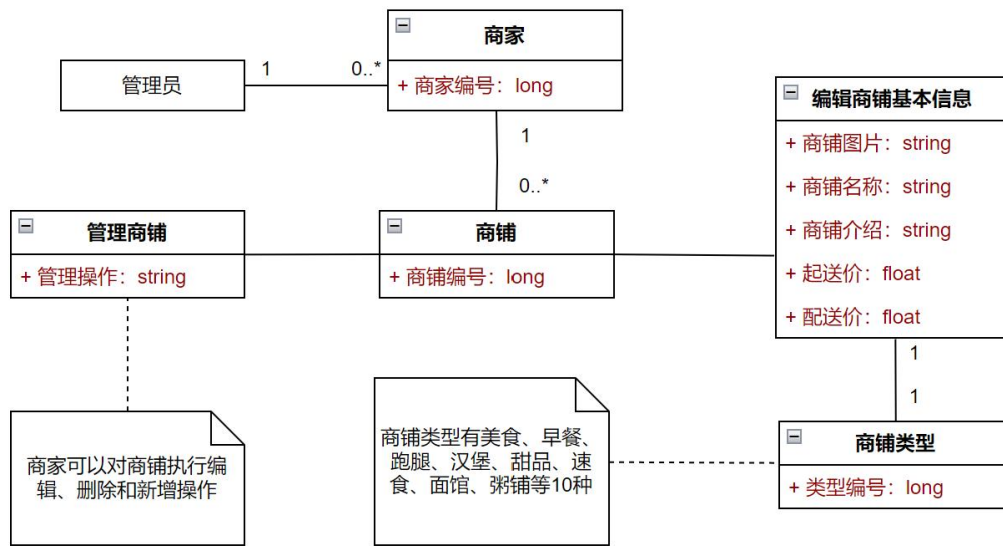


图 1-4 商铺管理类图

(5) 用户管理

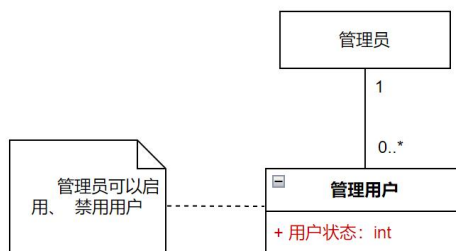


图 1-5 用户管理类图

(6) 点赞收藏评分

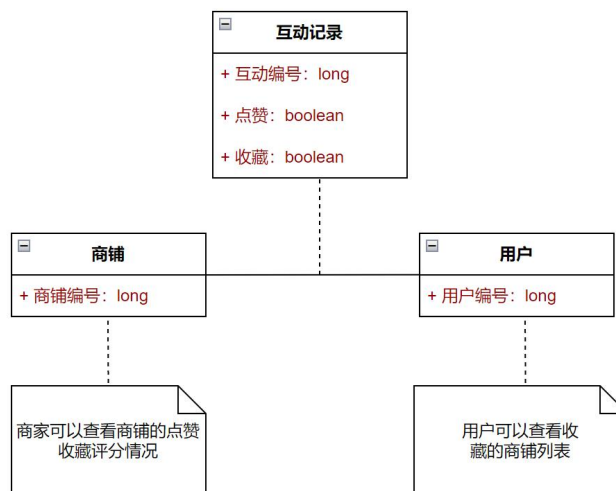


图 1-6 点赞收藏评分类图

1.3.2 业务流程分析 (UML活动图)

(1) 用户登录 & 注册流程

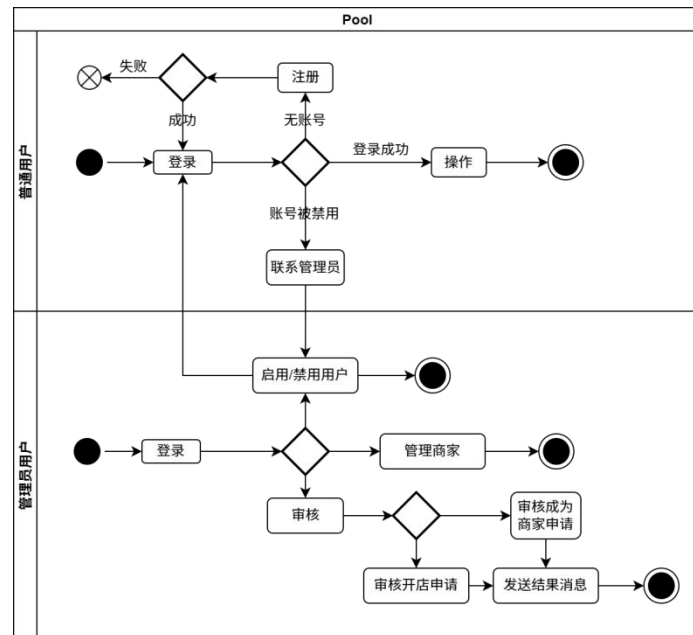


图 1-7 登录注册活动图

(2) 订单处理流程

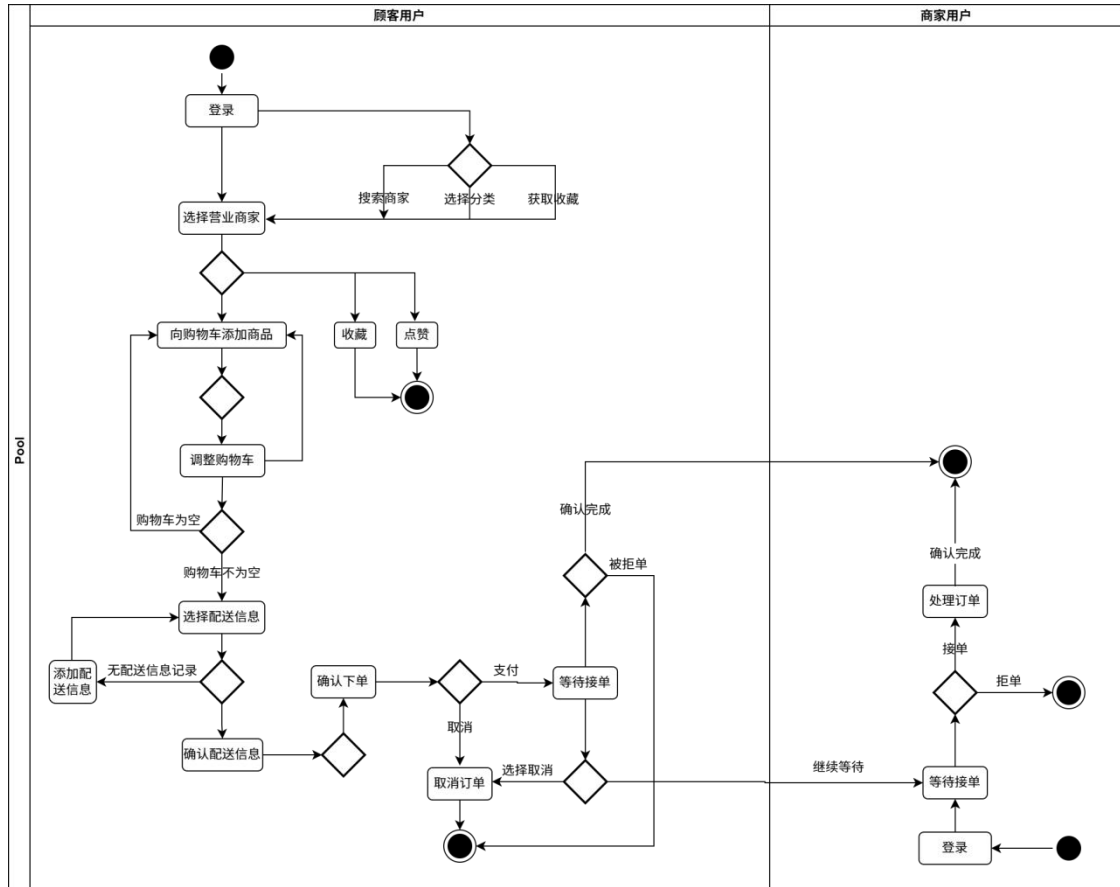


图 1-8 订单处理活动图

(3)商家管理流程

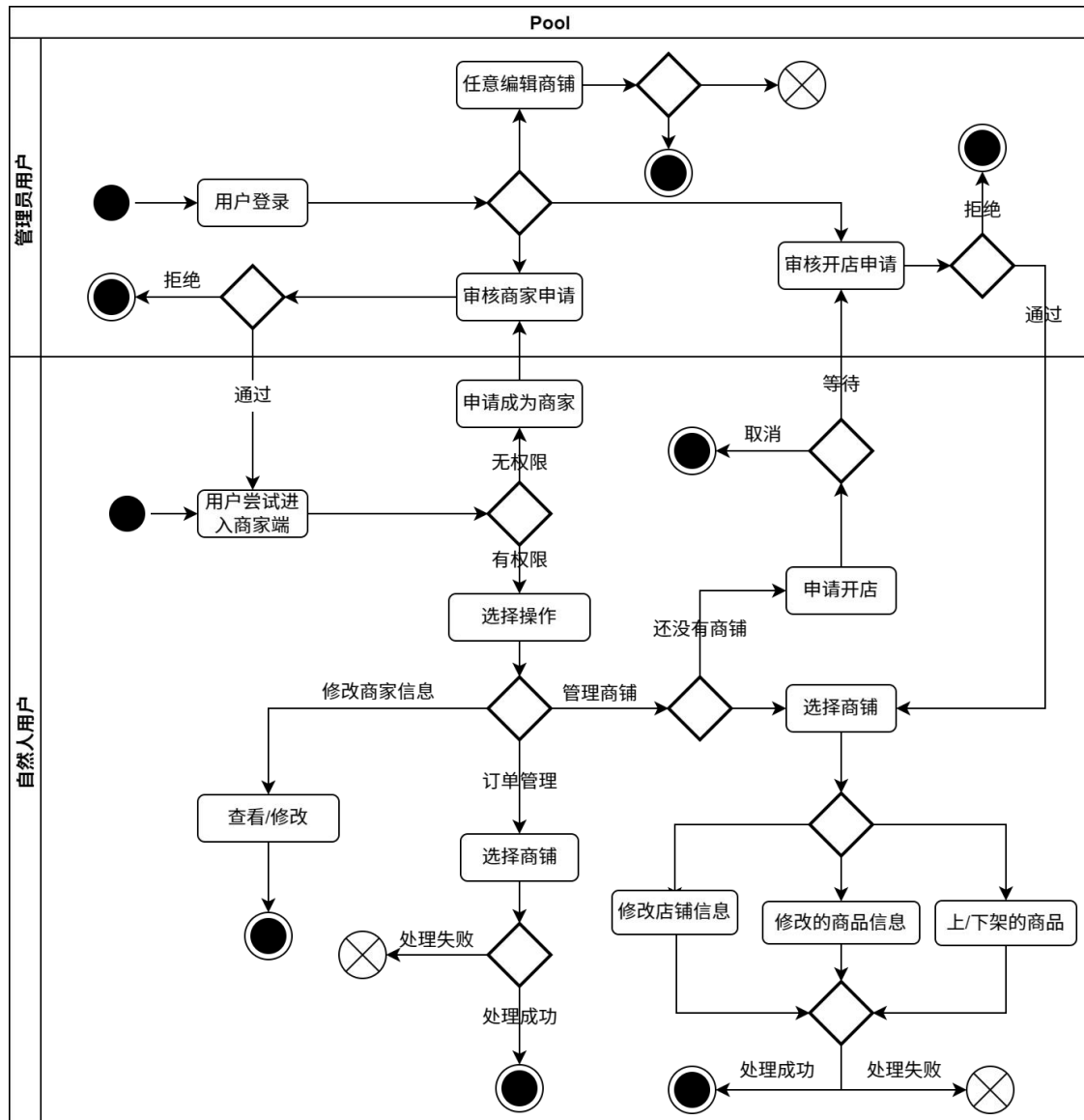


图 1-9 商家管理活动图

1.4 功能性需求

1.4.1 总用例

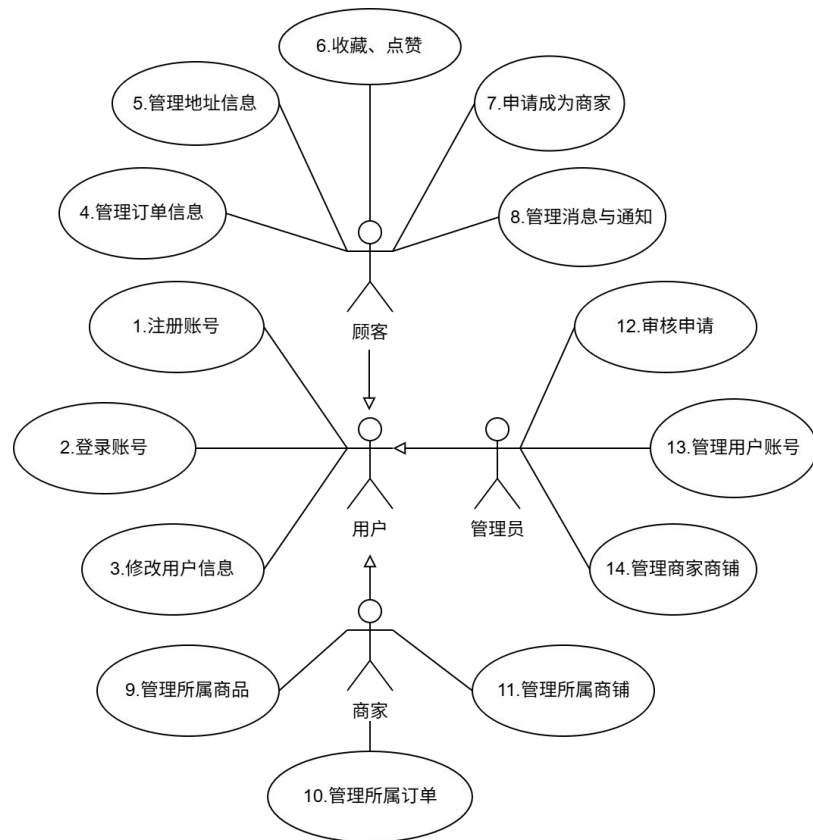


图 1-10 总用例图

1.4.2 用户的用例

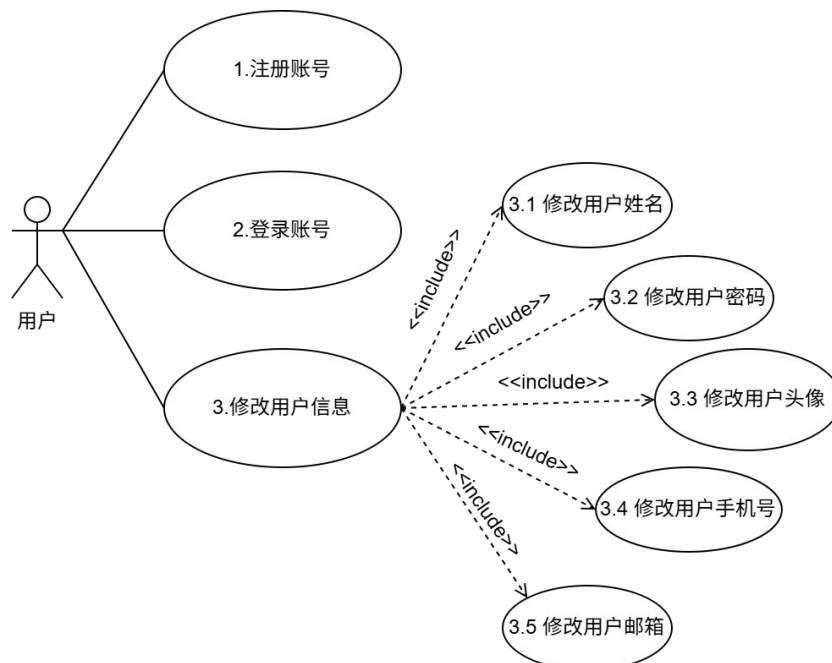


图 1-11 用户用例图

表 1-3 注册账号用例说明

编号	1	名称	注册账号
执行者	用户	优先级	高
描述	用户使用用户名进行账户注册, 设置账户密码		
前置条件	该用户名未被注册		
基本流程	1. 输入用户名 2. 输入密码 3. 输入其他个人信息		
结束状况	注册成功进入首页		
可选流程	-		
异常流程	1. 用户已经注册, 显示用户已存在		
备注	用户注册后即拥有顾客权限		

表 1-4 登录账号用例说明

编号	2	名称	登录账号
执行者	用户	优先级	高
描述	用户使用用户名密码进行账户登录		
前置条件	1. 提交正确的用户名 2. 提交正确的密码		
基本流程	1. 输入用户名 2. 输入密码 3. 显示登录成功信息		
结束状况	1. 登录成功后返回上一个页面 2. 登录成功后进入管理员首页		
可选流程	注册账号		
异常流程	1. 用户未注册进入用户注册流程 2. 密码验证不通过		
备注	只要该用户有管理员权限, 登录后便进入管理端首页否则进入顾客端首页		

表 1-5 修改用户信息用例说明

编号	3	名称	修改用户信息
执行者	用户	优先级	高
描述	用户修改用户信息		
前置条件	1.用户已登录		
基本流程	1. 进入“我的”界面 2. 修改用户姓名、密码、头像、手机号、邮箱 3. 提交修改		
结束状况	1. 显示修改后的用户信息		
可选流程			
异常流程	1. 用户修改后的手机号格式不符合规范 2. 用户修改后的邮箱格式不符合规范		
备注	-		

1.4.3 顾客的用例

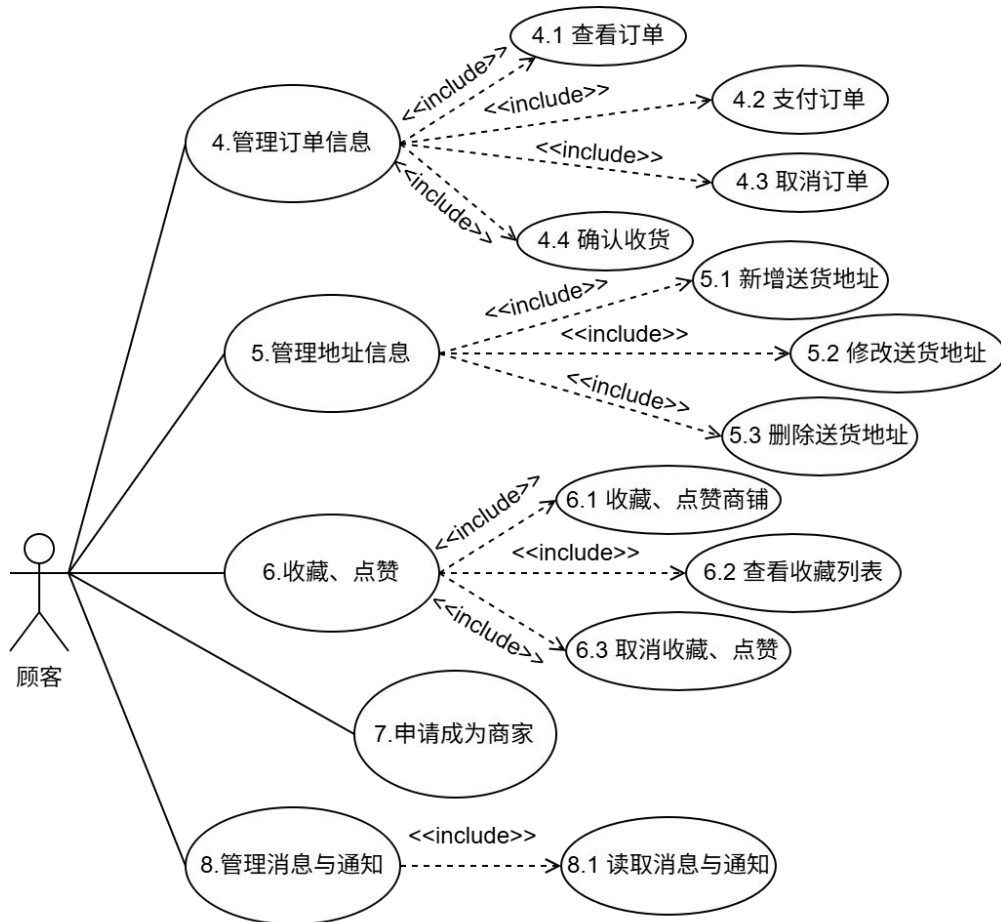


图 1-12 顾客用例图

表 1-6 查看订单说明

编号	4.1	名称	查看订单
执行者	顾客	优先级	低
描述	顾客查看订单		
前置条件	无		
基本流程	1. 点击页面下方“订单”图标 2. 查看订单列表 3. 点击可查看订单详情		
结束状况	显示订单列表		
可选流程	顾客可以选择查看不同状态的订单列表：待支付、待接单、已接单、已取消、已完成		
异常流程	-		
备注	-		

表 1-7 支付订单说明

编号	4.2	名称	支付订单
执行者	顾客	优先级	低
描述	顾客支付订单		
前置条件	顾客下单		
基本流程	1. 点击“去支付”按钮 2. 选择微信支付或者支付宝支付（默认）		
结束状况	订单已支付，状态变为待接单		
可选流程	-		
异常流程	-		
备注	-		

表 1-8 取消订单说明

编号	4.3	名称	取消订单
执行者	顾客	优先级	低
描述	顾客取消订单		
前置条件	订单未被商家接单		
基本流程	1. 顾客点击“取消订单”按钮 2. 在弹出的提示框中选择“确认”		
结束状况	订单状态变为已取消		
可选流程	-		
异常流程	订单已被商家接单，弹出提示框“订单已被接单，不能取消！”		
备注	顾客取消订单后商家端该订单状态显示同步被修改为“已取消”		

表 1-9 确认收货说明

编号	4.4	名称	确认收货
执行者	顾客	优先级	低
描述	顾客将订单状态改为已完成		
前置条件	订单已被商家接单		
基本流程	1. 点击“确认收货”按钮		
结束状况	订单状态变为已完成		
可选流程	-		
异常流程	-		
备注	顾客确认收货后商家端该订单状态显示同步被修改为“已完成”		

表 1-10 新增收货地址说明

编号	5.1	名称	新增送货地址
执行者	顾客	优先级	低
描述	顾客新增地址列表中的送货地址		
前置条件	-		
基本流程	1. 点击地址界面的新建用户地址 2. 依次填入地址信息 3. 显示新建地址成功界面		
结束状况	地址列表出现新地址		
可选流程	1. 地址中的必填信息未填写，需要写入		
异常流程	-		
备注	-		

表 1-11 修改收货地址说明

编号	5.2	名称	修改送货地址
执行者	顾客	优先级	低
描述	顾客修改地址列表中的送货地址		
前置条件	-		
基本流程	1. 点击地址界面的用户地址 2. 修改地址信息 3. 显示修改地址成功界面		
结束状况	地址列表出现修改后的新地址		
可选流程	地址中的必填信息未填写，需要写入		
异常流程	-		
备注	-		

表 1-12 删除收货地址说明

编号	5.3	名称	删除送货地址
执行者	顾客	优先级	低
描述	顾客删除地址列表中的送货地址		
前置条件	地址列表中有已经存在的地址		
基本流程	1. 点击地址界面的用户地址，进入用户的地址列表界面 2. 选中需要删除的地址，点击删除，询问是否删除 3. 显示删除地址成功界面		
结束状况	用户的地址列表界面原地址消失		
可选流程	误触点到删除，询问是否删除时点击否，返回用户地址列表界面		
异常流程			
备注			

表 1-13 收藏点赞店铺说明

编号	6.1	名称	收藏、点赞商铺
执行者	顾客	优先级	低
描述	顾客收藏、点赞商铺		
前置条件	无		
基本流程	1. 进入商铺 2. 点击收藏或点赞按钮		
结束状况	收藏或点赞按钮被点亮		
可选流程	-		
异常流程	-		
备注	-		

表 1-14 查看收藏列表说明

编号	6.2	名称	查看收藏列表
执行者	顾客	优先级	低
描述	顾客查看收藏列表		
前置条件	无		
基本流程	1. 进入“我的”界面 2. 点击我的收藏 3. 查看收藏的所有商铺，点击可进入		
结束状况	-		
可选流程	-		
异常流程	-		
备注	-		

表 1-15 取消点赞收藏说明

编号	6.3	名称	取消收藏、点赞
执行者	顾客	优先级	低
描述	顾客取消收藏、点赞		
前置条件	无		
基本流程	1. 进入商铺 2. 再次点击收藏或点赞按钮		
结束状况	收藏或点赞按钮变暗，取消收藏后该店铺不再出现在我的收藏		
可选流程	-		
异常流程	-		
备注	-		

表 1-16 申请成为商家说明

编号	7	名称	申请成为商家
执行者	顾客	优先级	低
描述	顾客申请成为商家		
前置条件	无		
基本流程	1. 进入“我的”页面 2. 点击申请成为商家按钮		
结束状况	管理端用户审核列表出现新条目，等待管理员审核		
可选流程	-		
异常流程	重复申请		
备注	-		

表 1-17 读取消息通知说明

编号	8.1	名称	读取消息与通知
执行者	顾客	优先级	低
描述	顾客读取消息与通知		
前置条件	无		
基本流程	1. 进入“我的”页面 2. 点击消息与通知 3. 点击待读取的消息		
结束状况	读取后消息与通知红点消失，消息由蓝色变为灰色		
可选流程	-		
异常流程	-		
备注	-		

1.4.4 商家的用例

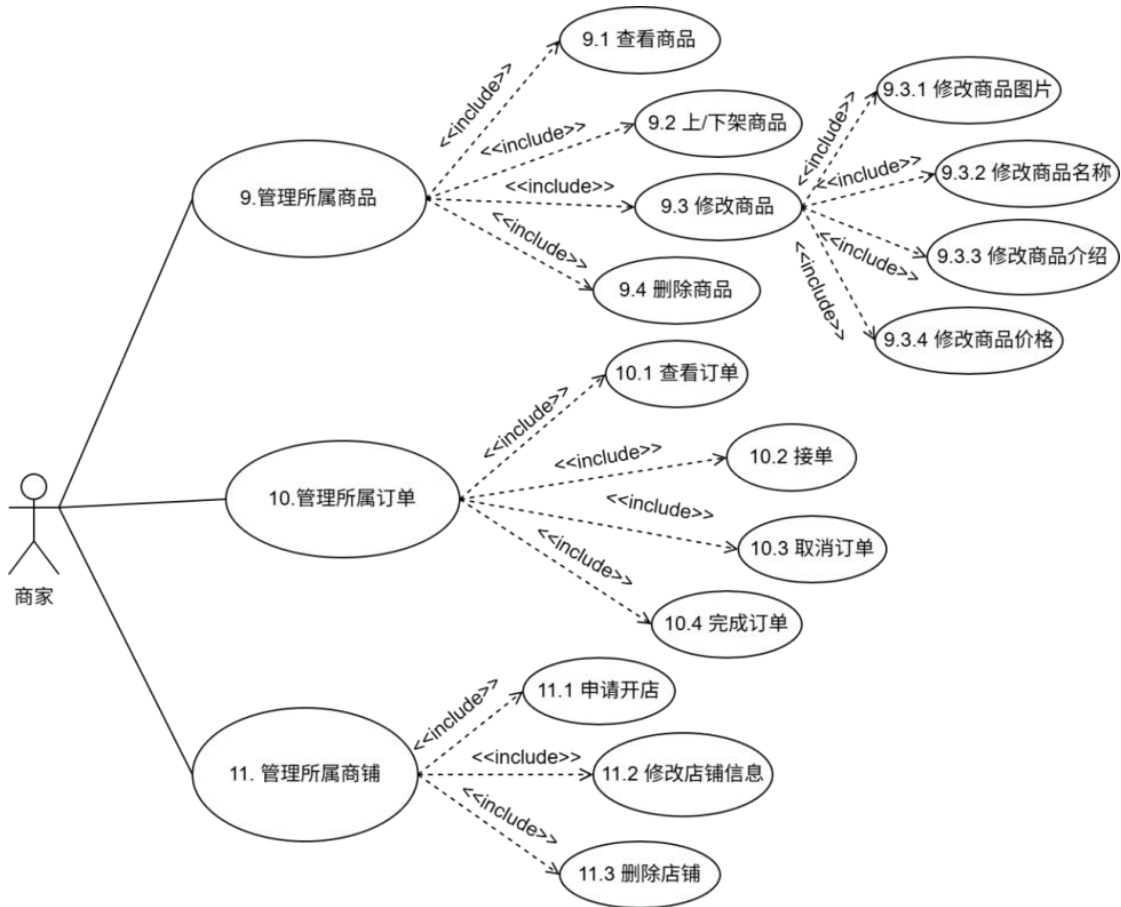


图 1-13 商家用例图

表 1-18 查看商品说明

编号	9.1	名称	查看商品
执行者	商家	优先级	低
描述	商家查看所售商品		
前置条件	无		
基本流程	1. 点击页面下方“商铺”图标 2. 点击对应商铺的“编辑”按钮		
结束状况	显示该商铺的所有商品		
可选流程	-		
异常流程	-		
备注	-		

表 1-19 上下架商品说明

编号	9.2	名称	上/下架商品
执行者	商家	优先级	低
描述	商家上/下架商品		
前置条件	无		
基本流程	1. 进入对应商铺 2. 点击对应商品的“上架/下架”按钮		
结束状况	下架商品，则该商品条目名称下方出现“已下架” 上架商品，则该商品条目恢复原状		
可选流程	-		
异常流程	-		
备注	-		

表 1-20 修改商品说明

编号	9.3	名称	修改商品
执行者	商家	优先级	低
描述	商家修改商品		
前置条件	无		
基本流程	1. 进入对应商铺 2. 点击对应商品的“编辑”按钮 3. 输入商品修改后的信息		
结束状况	商铺详情页面显示修改后的商品		
可选流程	-		
异常流程	-		
备注	修改项包括修改商品图片、名称、介绍、价格		

表 1-21 删除商品说明

编号	9.4	名称	删除商品
执行者	商家	优先级	低
描述	商家删除商品		
前置条件	无		
基本流程	1. 进入对应商铺 2. 点击对应商品的“删除”按钮 3. 输入商品修改后的信息		
结束状况	商铺详情页面显示修改后的商品		
可选流程	-		
异常流程	-		
备注	修改项包括修改商品图片、名称、介绍、价格		

表 1-22 查看订单说明

编号	10.1	名称	查看订单
执行者	商家	优先级	低
描述	商家查看本商户的订单		
前置条件	无		
基本流程	1. 进入商家“订单”页面 2. 选择对应商铺		
结束状况	显示商家该商铺不同状态的订单列表		
可选流程	-		
异常流程	-		
备注	商家端显示的订单状态分为：待接单、已接单、已取消、已完成		

表 1-23 接单说明

编号	10.2	名称	接单
执行者	商家	优先级	低
描述	商家接单		
前置条件	无		
基本流程	1. 进入商家“订单”页面，选择对应商铺 2. 选择查看“待接单”订单列表 3. 点击“接单”按钮		
结束状况	该订单从“待接单”状态转为“已接单”状态		
可选流程	-		
异常流程	-		
备注	当顾客支付订单后对应商家的待接单列表增加该订单；商家接单后顾客订单状态显示同步修改为“已接单”		

表 1-24 取消订单说明

编号	10.3	名称	取消订单
执行者	商家	优先级	低
描述	商家取消订单		
前置条件	无		
基本流程	1. 进入商家“订单”页面，选择对应商铺 2. 选择查看“待接单”订单列表 3. 点击“取消”按钮		
结束状况	该订单从“待接单”状态转为“已取消”状态		
可选流程	-		
异常流程	-		
备注	仅商家接单前可以取消订单		

表 1-25 完成订单说明

编号	10.4	名称	完成订单
执行者	商家	优先级	低
描述	商家完成订单		
前置条件	无		
基本流程	1. 进入商家“订单”页面，选择对应商铺 2. 选择查看“已接单”订单列表 3. 点击“完成”按钮		
结束状况	该订单从“已接单”状态转为“已完成”状态		
可选流程	-		
异常流程	-		
备注	商家完成订单后顾客对应的订单状态显示同步变为“已完成”		

表 1-26 申请开店说明

编号	11.1	名称	申请开店
执行者	商家	优先级	低
描述	商家申请开店		
前置条件	无		
基本流程	1. 点击“申请新店”按钮 2. 填写店铺信息		
结束状况	开店申请出现在管理端商铺审核列表，等待审核		
可选流程	-		
异常流程	-		
备注	-		

表 1-27 修改店铺信息说明

编号	11.2	名称	修改店铺信息
执行者	商家	优先级	低
描述	商家修改店铺信息		
前置条件	无		
基本流程	1. 点击对应店铺的“编辑”按钮 2. 点击“编辑商家信息” 3. 可修改店铺图片、名称、起送费、配送费、简介、类型		
结束状况	显示修改后的店铺		
可选流程	-		
异常流程	-		
备注	-		

表 1-28 删除店铺说明

编号	11.3	名称	删除店铺
执行者	商家	优先级	低
描述	商家删除店铺		
前置条件	无		
基本流程	点击对应店铺的“删除”按钮 在弹出弹窗中选择“确认删除”		
结束状况	显示删除该店铺后的店铺列表		
可选流程	-		
异常流程	-		
备注	-		

1.4.5 管理员的用例

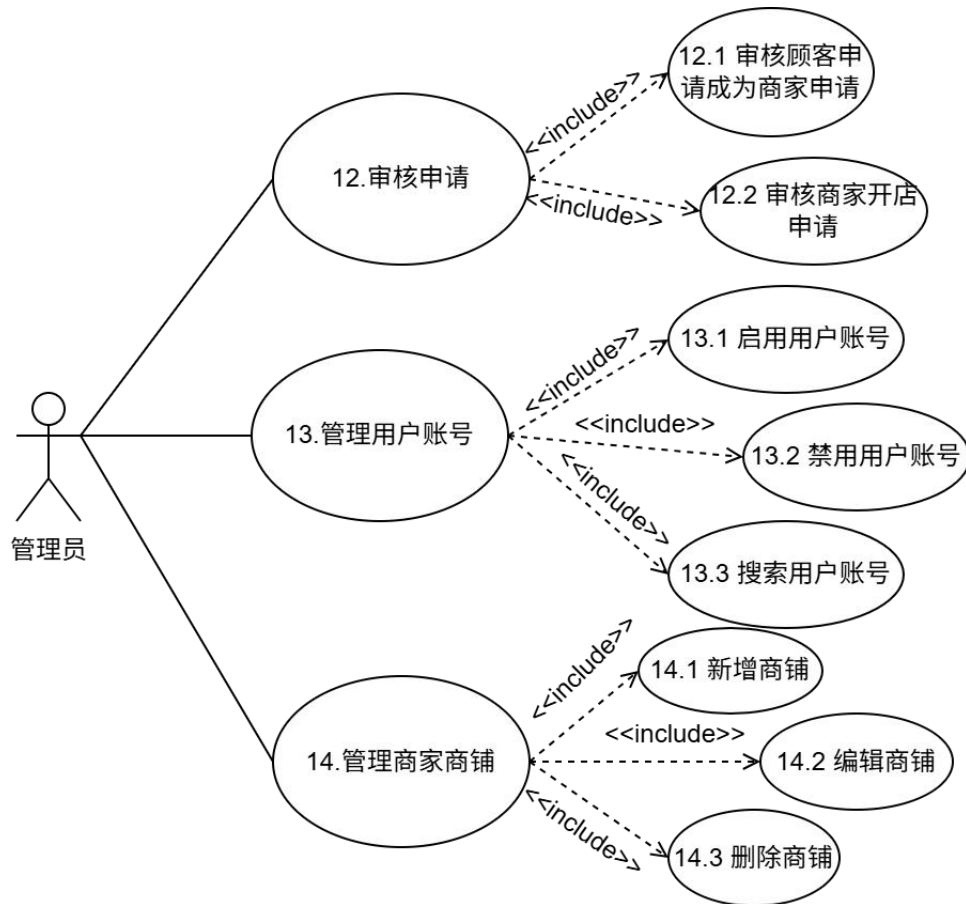


图 1-14 管理员用例图

表 1-29 成为商家申请说明

编号	12.1	名称	审核顾客申请成为商家申请
执行者	管理员	优先级	高
描述	管理员审核		
前置条件	无		
基本流程	进入用户审核列表，点击“审核”按钮，查看申请人详细信息选择“批准”或“拒绝”		
结束状况	顾客端“我的”页面的消息与通知出现新消息，通知审核结果；若管理员批准申请，顾客端“申请成为商家”按钮变为“切换到商家端”按钮		
可选流程	-		
异常流程	-		
备注	-		

表 1-30 审核商家开店申请说明

编号	12.2	名称	审核商家开店申请
执行者	管理员	优先级	高
描述	管理员审核		
前置条件	无		
基本流程	进入商铺审核列表，点击“审核”按钮，查看商铺详细信息选择“批准”或“拒绝”		
结束状况	顾客端“我的”页面的消息与通知出现新消息，通知审核结果；若管理员同意申请，商铺出现在商家商铺页面的已上线列表，否则出现在未通过列表		
可选流程	-		
异常流程	-		
备注	-		

表 1-31 启用用户账号说明

编号	13.1	名称	启用用户账号
执行者	管理员	优先级	高
描述	管理员启用用户账号		
前置条件	无		
基本流程	管理员进入“用户管理”页面 对于已禁用用户，点击“启用”按钮		
结束状况	该用户状态由“已禁用”变为“已启用”，出现在已启用用户列表		
可选流程	-		
异常流程	-		
备注	-		

表 1-32 禁用用户账号说明

编号	13.2	名称	禁用用户账号
执行者	管理员	优先级	高
描述	管理员禁用用户账号		
前置条件	无		
基本流程	管理员进入“用户管理”页面 对于已启用用户，点击“禁用”按钮		
结束状况	该用户状态由“已启用”变为“已禁用”，出现在已禁用用户列表		
可选流程	-		
异常流程	-		
备注	-		

表 1-33 搜索用户账号说明

编号	13.3	名称	搜索用户账号
执行者	管理员	优先级	高
描述	管理员搜索用户账号		
前置条件	无		
基本流程	1.管理员进入“用户管理”页面 2. 在搜索栏搜索用户名、手机号或邮箱		
结束状况	得到模糊匹配结果		
可选流程	-		
异常流程	-		
备注	-		

表 1-34 新增商铺说明

编号	14.1	名称	新增商铺
执行者	管理员	优先级	高
描述	管理员新增商铺		
前置条件	无		
基本流程	管理员进入“商铺管理”页面 进入对应商家 点击“新增商铺”按钮 填写新商铺的图片、名称、地址、简介、类型		
结束状况	该商家商铺列表出现新增的商铺		
可选流程	-		
异常流程	-		
备注	-		

表 1-35 编辑商铺说明

编号	14.2	名称	编辑商铺
执行者	管理员	优先级	高
描述	管理员编辑商铺		
前置条件	无		
基本流程	管理员进入“商铺管理”页面 进入对应商家，找到对应商铺 点击“编辑”按钮 可编辑商铺图片、名称、地址、简介、类型		
结束状况	该商家商铺列表出现编辑后的商铺		
可选流程	-		
异常流程	-		
备注	-		

表 1-36 删除商品说明

编号	14.3	名称	删除商铺
执行者	管理员	优先级	高
描述	管理员删除商铺		
前置条件	无		
基本流程	管理员进入“商铺管理”页面 进入对应商家，找到对应商铺 点击“删除”按钮 确认删除		
结束状况	该商家商铺列表不再出现该商铺		
可选流程	-		
异常流程	-		
备注	-		

1.4.6 其他功能性需求

首页提供搜索功能，用户可以搜索饿了么平台上的商铺；接入AI智能体，方便用户对商品的咨询；接入高德api，进行地理位置选择；首页轮播图展示销量前三的商铺；首页下方的综合排序按照商铺的评分（即商铺所获点赞数与收藏数加权归一的值）倒序排列；销量排序按照商铺的销量倒序排列；筛选可以按照是否免配送费、起送费的价格区间筛选相应的商铺。

1.5 非功能性需求

1.5.1 系统架构

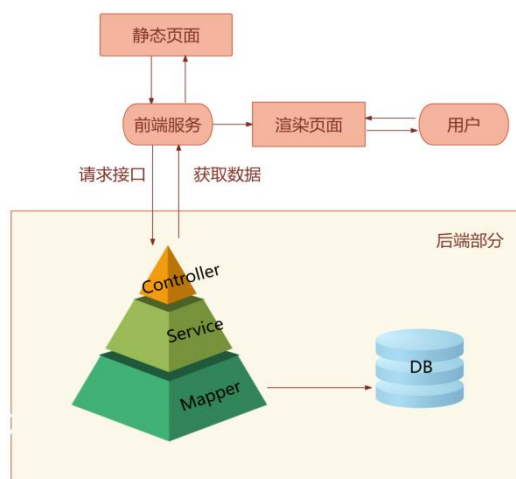


图 1-15 系统技术架构

1.5.2 安全性

饿了么在前后端采用多种方案共同实现对于用户敏感信息的保护：

- (1) 在传输密码时, 使用 HTTP POST 进行传输, 不在 URL 中传输敏感数据
 - (2) 后端使用 SpringSecurity+JWT 实现用户登录和认证
 - (3) 用户注册时, 后端利用 SpringSecurity 中的 BCryptPasswordEncoder 对密码进行加密处理之后再存入数据库, 有效防止数据库被入侵后导致用户密码信息的泄露风险
- 在前端, 我们在创建好的 axios 实例 axiosInstance 上, 挂载两个拦截器

(4) 请求拦截器:

- 1. 前端向后端发送请求之前拦截
- 2. 在 header 添加 token, 若有请求错误, 在控制台打出

(5) 响应拦截器:

- 1. 后端向前端发送响应, 前端接到之后拦截
- 2. 处理响应状态码错误 401, 500 等
- 3. 网络错误, 重试三次, 重试失败, 跳转回主页
- 4. 重试次数超过最大限制 3, 跳转回主页

第二章 项目设计文档

2.1 数据库设计

2.1.1 DB 一览表

注：NO 旁标注 '*' 的表为新添数据表。

表 2-1 数据库表一览

NO	数据库表名	中文概括名	说明
1*	ai_chat_history	AI对话历史表	接入智能体对话历史
2	authority	权限类型表	用户可以拥有的权限类型名
3	business	商铺表	商铺信息
4	cart	购物车表	用户购物车商品信息
5	delivery_address	配送地址表	用户配送地址信息
6	food	商品表	商铺中的商品信息
7*	merchant_interaction	用户对商铺动作表	用户点赞/收藏商铺记录
8*	notification	消息通知表	消息通知记录
9	orderdetail	订单详情表	订单中包含的商品信息
10	orders	订单表	订单信息
11*	permission_application	权限申请表	用户权限申请记录
12	person	自然人信息表	自然人(商家/顾客用户)的信息
13	user_authority	用户权限表	记录用户拥有的权限
14	users	用户表	用户(登录)信息

2.1.2 表结构

对于表约束，使用简写，说明如表2-2，NO 旁标注 '*' 的字段为新添字段。

表 2-2 约束说明

标识	英文	解释
PK	primary key	主键
FK	foreign key	外键
NN	not null	非空
UQ	unique	唯一索引
AI	auto increment	自增长列
DEL	delete	逻辑删除标记 0-正常 1-删除

1. ai_chat_history (AI对话历史表*)

表 2-3 AI对话历史表

NO	Field	Type	Size	Default	Constraint	Comment
1	id	bigint			NN PK AI	对话id
2	create_time	timestamp				创建时间
3	creator	bigint				创建用户id
4	is_deleted	tinyint	1	0	DEL	逻辑删除
5	update_time	timestamp				更新时间
6	updater	bigint				更新用户id
7	user_id	bigint			NN	用户id
8	session_id	varchar	64		NN	会话id
9	user_message	text			NN	用户提示词
10	ai_response	text			NN	AI回答内容
11	chat_type	varchar	20	general		询问类型
12	processing_time	bigint				过程耗时
13	context_data	json				上下文信息

2. authority (权限类型表)

表 2-4 权限类型表

NO	Field	Type	Size	Default	Constraint	Comment
1	name	varchar	50		PK NN	权限名

3. business (商铺表)

表 2-5 商铺表

NO	Field	Type	Size	Default	Constraint	Comment
1	id	bigint			PK AI NN	商铺id
2	create_time	timestamp				创建时间
3	creator	bigint				创建用户id
4	is_deleted	tinyint	1	0	DEL	逻辑删除
5	update_time	timestamp				更新时间
6	updater	bigint				更新用户id
7	business_address	varchar	255			商家地址
8	business_explain	varchar	255			商家介绍
9	business_img	text				商家图片
10	business_name	varchar	255		NN	商家名称
11	delivery_price	decimal	(10,2)		NN	配送费

12	order_type_id	int		1		点餐分类
13	remarks	varchar	255			备注
14	start_price	decimal	(10,2)		NN	起送费
15	user_id	bigint			NN FK(user.id)	商家用户id
16*	status	tinyint	0			状态 0-待审核 1-已上线

4. cart (用户购物车商品信息)

表 2-6 用户购物车商品信息表

NO	Field	Type	Size	Default	Constraint	Comment
1	id	bigint			PK AI NN	记录id
2	create_time	timestamp				创建时间
3	creator	bigint				创建用户id
4	is_deleted	tinyint	1	0	DEL	逻辑删除
5	update_time	timestamp				更新时间
6	updater	bigint				更新用户id
7	quantity	int			NN	商品数量
8	business_id	bigint			NN FK(business.id)	商品所属商铺id
9	customer_id	bigint			NN FK(user.id)	顾客id
10	food_id	bigint			NN FK(food.id)	商品id

5. delivery_address (配送地址表)

表 2-7 配送地址表

NO	Field	Type	Size	Default	Constraint	Comment
1	id	int			PK AI NN	地址id
2	create_time	timestamp				创建时间
3	creator	bigint				创建用户id
4	is_deleted	tinyint	1	0	DEL	逻辑删除
5	update_time	timestamp				更新时间
6	updater	bigint				更新用户id
7	address	varchar	255			地址
8	contact_name	varchar	255			联系人名字
9	contact_sex	int				联系人性别
10	contact_tel	varchar	255			联系人电话
11	user_id	bigint			NN FK(user.id)	所属用户id

6. food (商品表)

表 2-8 商品表

NO	Field	Type	Size	Default	Constraint	Comment
1	id	bigint			PK AI NN	商品id
2	create_time	timestamp				创建时间
3	creator	bigint				创建用户id
4	is_deleted	tinyint	1	0	NN	逻辑删除
5	update_time	timestamp				更新时间
6	updater	bigint				更新用户id
7	food_explain	varchar	255			商品介绍
8	food_img	text				商品图片
9	food_name	varchar	255		NN	商品名称
10	food_price	decimal	(10,2)		NN	商品价格
11	remarks	varchar	255			备注
12	business_id	bigint			NN FK(business.id)	所属商铺id
13*	shelve_status	tinyint		1		是否上架 0-下架 1-上架

7. merchant_interaction (用户对商铺动作表*)

表 2-9 用户对商家动作表

NO	Field	Type	Size	Default	Constraint	Comment
1	id	bigint			PK AI NN	记录id
2	user_id	bigint			NN FK(user.id)	用户id
3	merchant_id	bigint			NN FK(business.id)	商铺id
4	liked	tinyint	1	0		是否点赞 0-否 1-是
5	collected	tinyint	1	0		是否收藏 0-否 1-是
6	create_time	datetime				记录创建时间
7	update_time	datetime				记录更新时间

8. notification (消息通知表*)

表 2-10 消息通知表

NO	Field	Type	Size	Default	Constraint	Comment
1	id	bigint			PK AI NN	记录id
2	user_id	bigint			NN FK(user.id)	接收用户id
3	notification_type	tinyint			NN FK(business.id)	消息类型：0-商家申 请审核，1-开店申请 审核
4	notification_content	varchar	500			消息内容
5	audit_result	tinyint				审核结果：1-通过，2 -拒绝
6	is_read	tinyint		0		阅读状态：0-未读，1 -已读
7	create_time	datetime				消息创建时间
9	read_time	datetime				消息阅读时间
10	is_deleted	tinyint		0		逻辑删除

9. orders (订单详情表)

表 2-11 订单详情表

NO	Field	Type	Size	Default	Constraint	Comment
1	id	bigint			PK AI NN	商铺id
2	create_time	timestamp				创建时间
3	creator	bigint				创建用户id
4	is_deleted	tinyint	1	0	DEL	逻辑删除
5	update_time	timestamp				更新时间
6	updater	bigint				更新用户id
7	quantity	int			NN	商品数量
8	food_id	bigint			NN FK(food.id)	商品id
9	order_id	bigint			NN FK(orders.id)	订单id
10*	food_price	decimal	(10,2)		NN	商品当下价格

10. orders (订单表)

表 2-12 订单表

NO	Field	Type	Size	Default	Constraint	Comment
1	id	bigint			PK AI NN	商铺id
2	create_time	timestamp				创建时间
3	creator	bigint				创建用户id
4	is_deleted	tinyint	1	0	DEL	逻辑删除
5	update_time	timestamp				更新时间
6	updater	bigint				更新用户id
7	order_date	timestamp				下单时间
8	order_state	int		0		订单状态 0-待支付 1-待接单 2-已接单 3-已完成 4-已取消
9	order_total	decimal	(10,2)		NN	订单总价
10	business_id	bigint			NN FK(business.id)	商铺id
11	customer_id	bigint			NN FK(user.id)	下单顾客用户id
12	address_id	bigint			NN FK(delivery_addresses.id)	配送地址id
13*	delivery_price	decimal	(10,2)		NN	配送费

11. permission_application (权限申请表*)

表 2-13 权限申请表

NO	Field	Type	Size	Default	Constraint	Comment
1	id	bigint			PK AI NN	记录id
2	userId	bigint			NN FK(user.id)	用户id
3	status	tinyint		0		审核状态 0-未审核 1-同意 2-拒绝
4	is_deleted	tinyint		0		逻辑删除
5	update_time	timestamp				更新时间
6	create_time	timestamp				创建时间

12. person (自然人信息表)

表 2-14 自然人信息表

NO	Field	Type	Size	Default	Constraint	Comment
1	email	varchar	255			邮箱
2	first_name	varchar	255			姓
3	gender	varchar	255			性别
4	last_name	varchar	255			名
5	phone	varchar	255			手机号
6	photo	text				照片
7	id	bigint			NN FK(user.id)	用户id

13. user_authority (用户权限表)

表 2-15 用户权限表

NO	Field	Type	Size	Default	Constraint	Comment
1	user_id	bigint			NN FK(user.id)	用户id
2	authority_name	varchar	50		NN FK(authority)	权限名称

14. user (用户表)

表 2-16 用户表

NO	Field	Type	Size	Default	Constraint	Comment
1	id	bigint			PK AI NN	用户id
2	create_time	timestamp				创建时间
3	creator	bigint				创建用户id
4	is_deleted	tinyint	1	0	NN	逻辑删除
5	update_time	timestamp				更新时间
6	updater	bigint				更新用户id
7	activated	tinyint	1			是否被禁用 0-正常 1-禁用
8	password	varchar	100		NN	密码(加密后)
9	username	varchar	100		NN	用户名

2.2 后端接口设计

为方便响应处理与异常情况的判断，统一将接口响应的数据模型统一为 `HttpResult`，具体说明如表2-17。

表 2-17 `HttpResult` 包装类字段说明

字段名	类型	说明
<code>success</code>	<code>boolean</code>	是否出现异常
<code>code</code>	<code>string</code>	异常状态码
<code>data</code>	<code>object</code>	返回数据
<code>message</code>	<code>string</code>	异常信息

2.2.1 商铺管理部分接口设计

(1) 商铺管理基础接口

1. 根据 id 获取商铺信息

`/api/businesses/{id}`

请求方法: GET

参数: id 商铺id(路径参数)

返回值: `HttpResult<BusinessVO>` 商铺信息对象（包含商家信息）

2. 根据id修改商铺信息（覆盖）

`/api/businesses/{id}`

请求方法: PUT

参数: id 商铺id(路径参数), `BusinessUpdateDTO`修改后的商铺信息对象

返回值: `HttpResult<BusinessVO>` 修改后的商铺信息对象（包含商家信息）

3. 根据id删除商铺

`/api/businesses/{id}`

请求方法: DELETE

参数: id 商铺id(路径参数)

返回值: `HttpResult<BusinessVO>` 删除的商铺信息对象（包含商家信息）

4. 修改商铺信息（部分更新）

`/api/businesses/{id}`

请求方法: PATCH

参数: id 商铺id(路径参数), `BusinessUpdateDTO`修改后的商铺信息对象

返回值: `HttpResult<BusinessVO>` 修改后的商铺信息对象

5. 获取所有商铺

`/api/businesses`

请求方法: GET

参数: 无

返回值: `HttpResult<List<BusinessVO>>` 商铺数组

6.新增商铺

`/api/businesses`

请求方法: POST

参数: `BusinessDTO` 新增商家信息对象

返回值: `HttpResult<BusinessVO>` 新增的商家对象

(2) 商铺管理新增接口

1.商铺的搜索、筛选与排序

`api/business/search`

请求方式: GET

参数: `keyword`、`isScore`、`isSales`

返回值: 匹配到的商铺信息的数组

功能: 通过传入的关键字模糊匹配商铺名、商品简介、商铺地址搜索店铺,可传入字符串 `keyword`, `isScore` 为 0/1 表示是否按照评分排序, `isSales` 为 0/1 表示是否按照销售量排序

2.商家和管理员新增店铺（带状态）

`api/business/apply`

请求方式: POST

参数: `business`

返回值: `int`

权限验证: 验证该用户是否有商家或管理员权限

3.查询某商家特定状态的所有的店铺

`api/business/merchant`

请求方式: GET

参数: `userId`、`status`

返回值: 所有符合条件的商铺信息数组

功能: 根据商家id查询该商家拥有的所有店铺, 同时可以根据传入的`staus`字段查询到未审核、已审核、审核失败三种状态的商铺

4.查询所有激活的商家

`api/business/active`

请求方式: GET

参数: 无

返回值: 所有符合条件的商家数组

功能: 管理员查询所有商家集团及其基本信息

5.查询不同类型的店铺

`api/business/type`

请求方式: GET

参数: type (查询的类型id, 范围1~10)

返回值: List<Business>

功能: 查询十种类型的商铺

6.查询轮播图商铺

api/business/carousel

请求方式: GET

参数: 无

返回值: List<BusinessSearchVO>

功能: 返回销量前三的商铺及其基本信息

7.更新某店铺信息 (带状态)

api/business/own

请求方式: PUT

参数: id、BusinessUpdateDTO

返回值: BusinessVO

2.2.2 点赞收藏部分

1. 更新商铺点赞收藏等信息

/api/merchant/interaction/updata

请求方式: POST

参数: MerchantInteractionDTO传入用户id、商铺id、是否点赞或收藏

2.获取某用户收藏列表

/api/merchant/interaction/collections

请求方式: GET

参数: userId (用户 id, 路径参数)

返回值: 商铺对象的数组

功能: 供用户在我的界面查看自己的收藏商铺

3.获取某商铺点赞收藏总数与评分

/api/merchant/interaction/stats

请求方式: GET

参数: merchantId (商铺id, 路径参数)

返回值: 商铺点赞收藏总数及评分

功能: 查询某一商家的点赞收藏数及归一化后的评分

4. 获取某商家下的所有商铺的点赞收藏总数

/api/merchant/interaction/statsByUserId

请求方式: GET

参数: `userId` (路径参数)

返回值: 商铺点赞收藏总数及评分的数组

功能: 供商家查询自己的所有商铺的点赞收藏数和评分列表

2.2.2 用户模块接口设计

(1) 用户模块基础接口

1. 新增用户 (仅登录账号)

`/api/users`

请求方法: POST

参数: `UserCreateDTO` 请求体包含用户创建信息

返回值: `UserVO` 返回创建的用户信息

2. 顾客注册

`/api/register`

请求方法: POST

参数: `PersonCreateDTO` 请求体包含自然人用户创建信息

返回值: `HttpResult<PersonVO>` 返回创建的自然人用户信息

3. 新增自然人用户

`/api/persons`

请求方法: POST

参数: `PersonCreateDTO` 请求体包含自然人用户创建信息

返回值: `PersonVO` 返回创建的自然人用户信息

4. 修改密码

`/api/password`

请求方法: POST

参数: `LoginDTO` 请求体包含用户名和密码

返回值: `String` 返回操作结果

5. 获取当前登录用户

`/api/user`

请求方法: GET

返回值: `UserVO` 返回当前登录用户信息

6. 用户登录认证

`/api/auth`

请求方法: POST

参数: `LoginDTO` 请求体包含登录凭证

返回值: `JWTToken` 返回认证令牌

(2) 用户模块新增接口

1. 启用/禁用用户

/api/{username}/status

请求方法: PUT

参数: username 路径参数表示用户名, activated 查询参数表示是否激活

返回值: `HttpResult<String>` 返回操作结果消息

2. 切换用户启用/禁用状态

/api/{username}/status

请求方法: PATCH

参数: username 路径参数表示用户名

返回值: `HttpResult<UserVO>` 返回用户信息

3. 修改个人信息

/api/person/info

请求方法: PUT

参数: `PersonUpdateDTO` 请求体包含个人信息更新内容

返回值: `HttpResult<Person>` 返回完整的个人信息

4. 获取不同状态的自然人用户

/api/persons

请求方法: GET

参数: status 查询参数表示用户状态

返回值: `HttpResult<List<PersonVO>>` 返回自然人用户列表

5. 搜索用户

/api/persons/search

请求方法: POST

参数: `UserSearchDTO` 请求体包含搜索关键词和状态

返回值: `HttpResult<List<PersonVO>>` 返回搜索到的用户列表

6. 根据ID获取自然人属性

/api/personInfo

请求方法: GET

参数: id 查询参数表示用户ID

返回值: `HttpResult<Person>` 返回指定用户的完整个人信息

7. 删除用户

/api/{username}

请求方法: DELETE

参数: username 路径参数表示用户名

返回值: `HttpResult` 返回操作结果

2.2.3 地址模块接口设计

(1) 地址模块基础接口

1. 新增地址

/api/addresses

请求方法: POST

参数: AddressCreateDTO 请求体包含地址创建信息

返回值: JsonResult<AddressVO> 返回创建的地址信息

(2) 地址模块新增接口

1. 删除配送地址

/api/addresses/removeDeliveryAddress

请求方法: POST

参数: DeliveryAddress 请求体包含地址信息

返回值: JsonResult 返回操作结果

2. 更新配送地址

/api/addresses/updateDeliveryAddress

请求方法: POST

参数: DeliveryAddress 请求体包含地址更新信息

返回值: JsonResult 返回操作结果

3. 获取用户地址列表

/api/addresses/listDeliveryAddressByUserId

请求方法: GET

参数: userId 查询参数表示用户ID

返回值: JsonResult<List<DeliveryAddress>> 返回地址列表

4. 根据ID获取配送地址

/api/addresses/getDeliveryAddressById

请求方法: POST

参数: DeliveryAddress 请求体包含地址ID

返回值: JsonResult<DeliveryAddress> 返回地址详细信息

2.2.4 权限申请管理接口设计

1. 审核商家申请

/api/permission/audit

请求方法: POST

参数: AuditPermissionDTO 请求体包含审核信息

返回值: JsonResult<PermissionApplication> 返回审核结果

2.审核开店申请

/api/permission/audit-shop

请求方法: POST

参数: BusinessPermissionDTO请求体包含店铺审核信息

返回值: JsonResult<BusinessPermissionVO> 返回审核结果

3.申请开店

/api/permission/apply-shop

请求方法: POST

参数: BusinessPermissionDTO请求体包含开店申请信息

返回值: JsonResult<BusinessPermissionVO> 返回申请结果

4.申请成为商家

/api/permission/apply-merchant

请求方法: POST

返回值: JsonResult<PermissionApplication> 返回申请结果

5.获取开店申请待审核列表

/api/permission/shop-applications

请求方法: GET

返回值: JsonResult<List<BusinessPermissionVO>> 返回开店申请列表

6.获取商家申请待审核列表

/api/permission/merchant-applications

请求方法: GET

返回值: JsonResult<List<MerchantApplicationsVO>> 返回商家申请列表

2.2.5 管理员模块接口设计

1.获取总用户数

/api/admin/countUser

请求方法: GET

返回值: JsonResult<Integer> 返回用户总数

2.获取总营业额

/api/admin/countPrice

请求方法: GET

返回值: JsonResult<Double> 返回总营业额

3.获取总店铺数

/api/admin/countBusiness

请求方法: GET

返回值: JsonResult<Integer> 返回店铺总数

2.2.6 消息与通知模块接口设计

1. 标记消息为已读

/api/notifications/{id}/read

请求方法: PUT

参数: id 路径参数表示消息ID

返回值: JsonResult 返回操作结果

2. 获取用户消息列表

/api/notifications

请求方法: GET

参数: userId 查询参数表示用户ID

返回值: JsonResult<List<Notification>> 返回消息列表

2.2.7 文件上传模块接口设计

1. 上传文件

/upload

请求方法: POST

参数: file 查询参数表示上传的文件

返回值: JsonResult<String> 返回文件访问路径

2.2.8 商品模块接口设计

(1) 基础接口

1. 根据商家或订单获取商品列表

/api/foods

请求方法: GET

参数: business 商铺id*, order 订单id*

返回值: JsonResult<List<FoodVO>> 商品信息列表

2. 根据订单id获取商品信息

/api/foods/{id}

请求方法: GET

参数: id 商品id

返回值: JsonResult<FoodVO> 商品信息对象

3. 新增商品

/api/foods

请求方法: POST

参数: foodDTO 商品信息对象

返回值: `HttpResult<FoodVO>` 商品id

4.修改商品信息

`/api/foods/{id}`

请求方法: PATCH

参数: `foodDTO` 商品信息对象, `id` 商品id

返回值: `HttpResult<FoodVO>` 修改成功后的商品对象

(2) 新增接口

1.根据商家获取商品列表 (普通用户只能看到已上架的)

`/api/foods/list`

请求方法: GET

参数: `businessId` 商铺id, `shelveStatus` 商品状态(0-未上架, 1-已上架)*

返回值: `HttpResult<List<FoodItemVO>>` 商品信息列表

2.商铺新增商品 (管理员可以随便添加,商家只能为自己的商铺添加)

`/api/foods/addItem`

请求方法: POST

参数: `foodCreateDTO` 新增商品信息对象

返回值: `HttpResult<Long>` 商品id

3.上架/下架商品 (商家用户只能操作自己的商铺)

`/api/foods/status`

请求方法: GET

参数: `file` 查询参数表示上传的文件

返回值: `HttpResult<Long>` 操作成功的商品id

4.商铺修改商品 (商家用户只能操作自己的商铺)

`/api/foods/modifyItem`

请求方法: POST

参数: `foodUpdateDTO` 修改商品对象

返回值: `HttpResult<Long>` 操作成功的商品id

5.商家删除商品 (商家用户只能操作自己的商铺)

`/api/foods/delete`

请求方法: GET

参数: `foodId` 商品id

返回值: `HttpResult<Long>` 操作成功的商品id

2.2.9 购物车模块接口设计

(1) 基础接口

1.向购物车添加商品

/api/carts

请求方法: POST

参数: cartItemCreateDTO 购物车对象

返回值: JsonResult<CartVO> 插入的购物车对象

(2) 新增接口

1.获取用户在指定商家的购物车商品列表

/api/carts/list

请求方法: GET

参数: businessId 商铺id

返回值: JsonResult<List<CartItemVO>> 用户在指定商铺的购物车列表

2.向购物车添加商品

/api/carts/add

请求方法: GET

参数: foodId 商品id, quantity 商品数量

返回值: JsonResult<Long> 成功插入的记录id

3.商家修改购物车指定商品数量

/api/carts/quantity

请求方法: GET

参数: cartId 记录id, quantity 商品数量

返回值: JsonResult<Long> 操作成功的记录id

4.清空用户在指定商家的购物车

/api/carts/clear

请求方法: GET

参数: businessId 商铺id

返回值: JsonResult<Long> 操作成功的记录id

5.移除指定购物车商品

/api/carts/remove

请求方法: GET

参数:cartId 记录id

返回值: JsonResult<Long> 操作成功的记录id

2.2.10 订单模块接口设计

(1) 基础接口

1.根据id获取用户订单

/api/orders/{id}

请求方法: GET

参数:cartId 记录id

返回值: JsonResult<OrderVO> 订单详情对象

2.新增订单

/api/orders

请求方法: POST

参数: orderDTO订单信息对象

返回值: JsonResult<OrderVO> 新增的订单id

(2) 新增接口

1.根据商家和状态获取订单列表

/api/orders/list/business

请求方法: GET

参数: businessId 商铺id*, orderState 订单状态(0-未支付 1-待接单 2-已接单 3-已完成 4-已取消)*

返回值: JsonResult<List<OrderItemDetailVO>> 订单信息列表

2.获取用户自己的相应状态的订单列表

/api/orders/list/user

请求方法: GET

参数: orderState 订单状态(0-未支付 1-待接单 2-已接单 3-已完成 4-已取消)*

返回值: JsonResult<List<OrderItemVO>> 订单信息列表

3.获取订单详情

/api/orders/detail

请求方法: GET

参数: orderId订单id

返回值: JsonResult<OrderItemDetailVO> 订单详细信息对象

4.设置订单状态

/api/orders/status

请求方法: PUT

参数: orderState 订单状态(0-未支付 1-待接单 2-已接单 3-已完成 4-已取消), orderId订单id

返回值: JsonResult<Long> 操作成功的订单id

5. 下单

/api/orders/submit

请求方法: GET

参数: businessId 商铺id, addressId 配送信息id

返回值: JsonResult<Long> 订单id

2.3 前端设计

饿了么的用户界面和用户体验设计围绕简洁、高效、用户友好等核心原则，旨在为消费者用户提供流畅的点餐、支付等体验，为商家用户提供了完整的出餐、查看订单状态等功能。通过清晰的布局、流畅的导航和实时反馈, 饿了么有效提升了用户点餐、支付、追踪订单的整体体验，符合时代下快节奏的生活方式和需求。

2.3.1 普通用户

(1) 首页设计

1. 整体布局

首页采用分层式布局设计，以白、蓝色为背景主色调，搭配品牌色作为重点标识和操作按钮色彩，营造轻快、高效的视觉氛围。页面顶部固定搜索栏，中部为分类导航网格，下部以轮播图形式动态展示销量榜单和推荐商家，布局层次清晰，符合用户自上而下的浏览习惯，确保核心功能一目了然。

2. 核心功能区域设计

顶部搜索框预设提示语“搜索饿了么商家、商品名称”，支持关键词搜索，提升搜索效率。分类导航区采用图标与文字结合的形式，涵盖美食、早餐、跑腿代购等高频场景，图标设计简洁直观，支持点击详情浏览。销量冠军榜模块通过动态数据加载，突出显示最受欢迎的商家，并标注配送点赞、收藏、费用和起送价，辅助用户快速决策。

3. 交互与反馈机制

用户点击分类图标或搜索框时，页面提供微动效反馈（如颜色高亮），增强操作感知。搜索过程中，系统实时返回匹配结果，确保内容相关性。

4. UI/UX设计特点：

1) 直观导航体系

通过网格化分类和滚动榜单，减少用户认知负荷，实现一键直达目标内容。个性化内容推荐基于算法动态调整商家展示顺序，提升用户黏性和转化率。高效搜索体验结合智能联想和历史记录，缩短用户路径。

2) 高效的导航设计

顶部搜索栏置于醒目位置，支持快速查找。横向分类导航提供11个常用品类入口，覆盖主要消费场景。底部导航栏固定显示核心功能模块，确保用户随时可访问关键页面。

3) 用户导向的内容布局

内容布局采用默认排序，也可以通过距离和销量进行筛选，让用户更方便的实现选择。商家信息卡片呈现起送价、配送费等关键决策信息，减少用户选择时间。

4) 简洁直观的操作路径

界面设计注重操作效率，重要功能按钮设计醒目且易于点击。排序和筛选功能置于明显位置，支持用户多维度浏览内容。

(2) AI智能体（小饿AI助手）界面设计

1. 整体设计

这是一个悬浮在右下角的聊天组件，当用户点击发送消息时，先把用户消息添加到界面上，然后调用后端 API，等 AI 回复后再添加到消息列表并自动滚动到底部。实现了一个关键的意图识别功能，通过检测用户输入的关键词来判断是商家咨询、菜品推荐还是订单相关的问题，然后传给后端不同的 chatType 参数。比如用户说"推荐个餐厅"，就识别为 business 类型；说"有什么好吃的"就是 food 类型。

在错误处理方面，做了多层次的降级策略。如果网络连接失败，会返回"网络连接失败，请检查网络后重试"这样的友好提示；如果后端AI服务不可用，就显示"AI服务暂时不可用，请稍后重试"。这样确保用户不管什么情况都能得到合理的反馈。

添加了打字指示器，就是那个三个小点的动画效果，让用户感觉AI真的在思考。还有快捷问题按钮，用户点击就能直接发送预设的问题，不用手动输入。另外还实现了对话历史功能，用户可以查看之前的聊天记录，甚至重新加载某个历史会话继续对话。

在界面设计上，用了渐变色和圆角设计让界面看起来更现代，并且做了响应式适配，在手机上聊天窗口会占满屏幕，在电脑上就是固定尺寸的浮窗。我还加了很多细节，比如消息发送时的滑入动画，按钮悬停的反馈效果，输入框的字符计数提示等等。

2. UI/UX设计特点：

1) 自然语言交互优化

对话流设计完全符合日常交流习惯，采用对话气泡式界面清晰区分用户与AI的消息。系统支持多轮上下文记忆能力，能够准确理解指代关系，避免用户重复提问。输入框配备智能联想功能，在用户输入过程中提供建议性问题，显著降低输入成本。

2) 主动式功能引导

界面通过预设高频问题按钮实现主动服务引导，直观展示AI的核心能力范围，有效降低用户首次使用时的认知门槛。智能推荐算法基于用户历史查询和场景时间自动生成个性化建议，如在餐点时间优先推荐美食相关功能。视觉设计上采用色彩对比突出核心操作区域，引导用户自然完成交互流程。

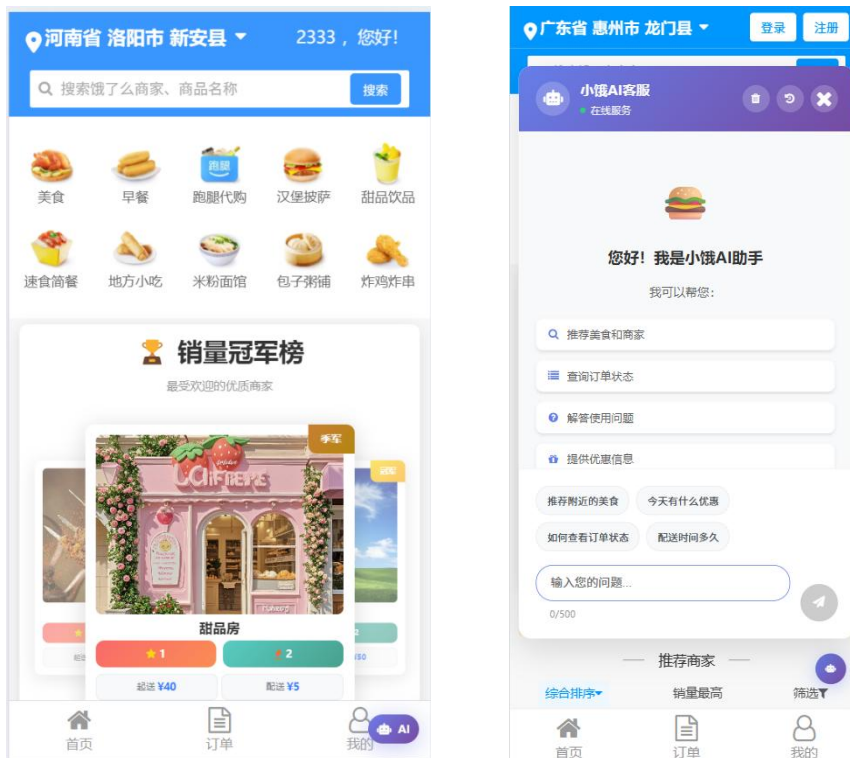


图 2-1

(3) 用户登录界面设计

1. 整体布局

登录界面采用极简主义设计风格，以纯白色为背景主色调，搭配深灰色文字和品牌蓝色操作按钮，形成清晰的视觉层次。页面布局采用绝对居中定位，登录表单以卡片形式悬浮于页面中央，四周留有适量留白，营造专注的登录氛围，减少视觉干扰。

2. 表单区域设计

顶部居中设置"用户登录"标题，使用加粗字体突出功能定位。用户名输入框限制长度，密码输入框采用密文显示模式。界面提供"记住我"复选框选项，默认未勾选状态，平衡便捷性与安全性需求。登录按钮采用品牌蓝色填充设计，色彩鲜明且具有足够的点击区域。

3. 验证与反馈机制

系统实施前端实时验证机制，用户名输入框自动过滤非法字符，密码框实时检测输入强度。当焦点离开输入框时立即触发格式验证，用户名格式错误时提示"登陆失败"，密码不符合要求时显示"密码错误"。登录按钮在通过基础验证前保持半透明禁用状态，通过后变为完全激活状态。

4. UI/UX设计特点：

1) 智能输入优化

用户名输入框支持自动记忆和补全功能，优先显示最近使用的账号。密码输入提供可视化的强度提示条，随输入内容动态变化颜色。界面支持键盘快捷操作，Enter键直接触发登录流程。

2) 安全与便捷平衡

通过"记住我"选项提供个性化记忆功能，在保障安全的前提下提升重复登录效率。密码采用对称加密传输，输入过程中实时掩码显示。错误提示信息避免具体技术细节，使用友好易懂的表述方式。

3) 响应式交互设计

登录过程提供明确的加载状态指示，成功跳转前显示进度动画。错误情况提供具体修正指引，连续失败时触发安全验证升级机制。界面元素适配不同屏幕尺寸。



图2-2

(4) 商家详情页面设计

1. 整体布局

商家详情页采用垂直滚动布局，顶部固定商家基础信息栏，中部为商品列表展示区，底部悬浮购物车结算栏。页面以白色为背景主色调，功能按钮突出显示，形成清晰的视觉动线，引导用户自上而下完成浏览和加购操作。

2. 核心信息展示设计

顶部商家信息栏清晰展示品牌名称，并突出显示起送价和配送费的关键决策信息。商家描述采用浅灰色小号字体，提供情感化表达。商品列表采用卡片式设计，左侧商品图右侧文字描述，商品名称标题突出，副标题和价格分层展示。

3.交互与购物车设计

底部悬浮购物车始终可见，当前金额未达起送价状态灰色显示，动态提示"另需配送费"和"¥xx起送"要求。用户添加商品时购物车实时更新金额，达到起送门槛后按钮变为激活状态。商品卡片点击区域充足，支持直接加购操作。

4.UI/UX设计特点：

1) 结构化信息呈现

关键决策信息（起送价、配送费）优先展示，减少用户计算成本。商品信息采用标准化的卡片布局，保证浏览效率。价格标签始终保持醒目位置，强化价值感知。

2) 渐进式购物引导

底部购物车提供实时反馈，清晰显示当前状态与起送门槛的差距。商品添加过程伴有微动效确认，增强操作感知。金额未达标时按钮保持禁用状态，避免无效下单。

3) 情景化卖点展示

商家描述增强亲和力，商品副标题突出口味特征。页面间距设置合理，保证信息密度适中，关键操作按钮尺寸符合手指操作规范，提升移动端使用体验。

(5) 商家列表页面设计

1. 整体布局

商家列表页采用信息流布局，顶部固定搜索栏与排序筛选栏，中部为无限滚动的商家信息流。页面以白色为背景，商家卡片间用细分割线区分，底部导航栏固定显示首页、订单、我的和AI入口。整体设计确保用户在浏览大量商家时能快速定位和筛选，信息层级清晰明确。

2. 核心信息展示设计

搜索栏保留全局搜索入口，排序筛选区提供“综合排序”、“销量最高”和“筛选”三个核心选项，以标签式按钮呈现当前选中状态。每个商家卡片集中展示关键决策信息：商家名称、评分、月售量、起送价与配送费，使用数字突出销量和价格，评分以分数形式精确显示。

3. 交互与筛选机制

用户点击排序选项时，页面即时刷新列表，选中状态以品牌色高亮显示。商家卡片支持点击进入详情页，下滑加载更多时提供加载动画反馈。筛选功能预计展开浮层提供多维度选择。底部导航栏的AI入口以图标形式集成，提供智能推荐快捷通道。

4. UI/UX设计特点：

1) 高效的信息密度管理

商家卡片布局紧凑，关键数据突出显示，减少用户认知负荷。评分精确到小数点后两位，增强信息可信度。

2) 灵活的排序与筛选

提供多维度排序方式，满足不同场景下的用户需求。筛选功能预留扩展性，支持按价格、距离、品类等条件细化搜索结果。交互反馈即时，提升筛选操作效率。

3) 情景化商家展示

评分与销量并重，辅助用户决策。起送价和配送费明确标注，避免结算时的意外成本。整体设计兼顾浏览效率与决策支持，适配用户在外卖场景下的快速选择习惯。

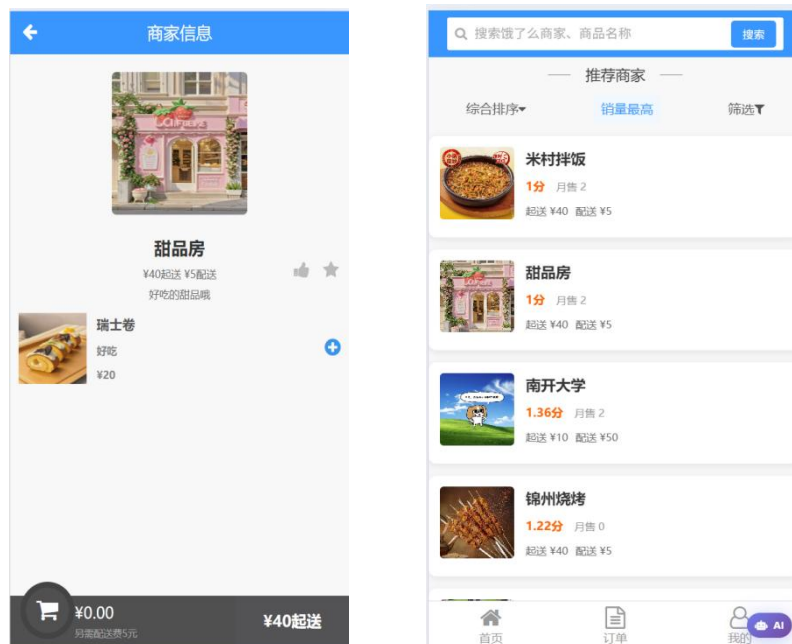


图2-3

(6) 订单配送地址选择页面设计

1. 整体布局

订单配送地址选择页面采用列表式布局，顶部通栏明确标示"订单配送地址"页面标题，中部区域展示已保存的收货地址列表，底部固定操作区域包含"新增收货地址"与"确认下单"两个核心按钮。页面背景采用浅灰色系，地址条目以白色卡片形式呈现，形成清晰的视觉层级，确保用户能够快速聚焦于地址选择任务。

2. 核心信息展示设计

每个地址条目独立成卡，优先展示收件人姓名与手机号码，下方为详细的地址文本。信息排版紧凑，关键识别信息（姓名、电话）突出显示。页面底部明确区分操作类型，"新增收货地址"为次要按钮使用线框样式，"确认下单"为主要按钮采用品牌填充色，视觉权重分明。

3. 交互与选择机制

用户通过直接点击地址卡片进行选择，被选中的地址卡片应有明确的视觉反馈，如边框色变化或背景色加深。点击"新增收货地址"将跳转至地址编辑页面，支持新增

后返回并自动刷新列表。"确认下单"按钮在用户至少选择一个有效地址后方可激活，确保流程的完整性。

4.UI/UX设计特点:

1)高效的选择流程

默认高亮显示最近使用或默认地址，减少用户操作步骤。地址信息布局直观，支持快速扫描与比较。清晰的按钮层次引导用户自然完成"选择-确认"或"新增-选择-确认"的操作路径。

2)灵活的管理入口

将"新增收货地址"入口与地址列表并列放置，既满足了即时添加新地址的需求，又避免了与选择操作的混淆。设计为后续地址编辑与管理功能的扩展预留了界面空间。

3)明确的操作引导

通过色彩与样式对比强化主次操作按钮的区别，引导用户最终完成下单确认。页面流程线性且封闭，有效防止用户在未完成地址选择的情况下误操作跳出，提升任务完成率。

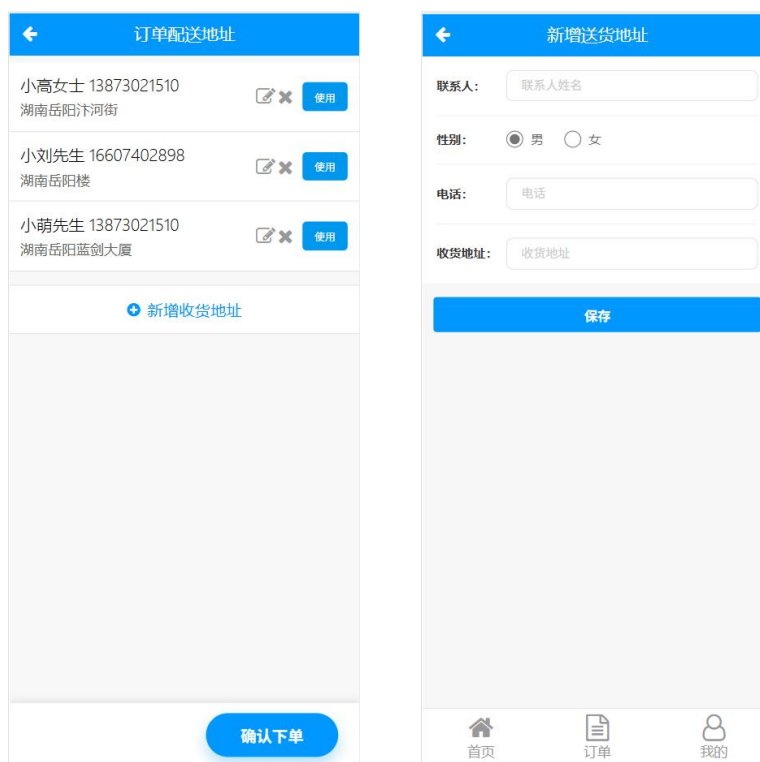


图2-4

(7) 确认订单页面设计

1. 整体布局

在线支付页面采用清晰的模块化布局，以白色背景为基础，通过分段式设计将不同信息区块有序组织。页面顶部明确标示"在线支付"标题，下方依次排列订单基本信息、订单详情、费用摘要和支付方式选择四大模块，最后以突出的底部操作栏收尾。

模块间使用分割线和间距进行视觉区分，营造出结构化、可预测的浏览路径，引导用户逐步确认信息并完成支付。

2. 核心信息展示设计

订单基本信息模块简洁展示收货人、地址和电话关键配送信息。订单详情模块突出商家名称和商品清单，商品名称与价格、数量分行明确标示。费用摘要部分将配送费单独列出，并与商品金额形成清晰合计。支付方式选项以列表形式呈现，“支付宝”与“微信支付”并列供用户选择。底部“确认支付”按钮以大字号和醒目色彩突出显示总金额，确保关键操作无疑虑。

3. 交互与反馈机制

用户选择支付方式时，界面提供即时视觉反馈，如单选按钮的选中状态变化。在用户点击“确认支付”按钮后，页面应出现明确的加载状态提示，并向用户传递支付处理中的信息。若支付方式未选择或支付过程遇到问题，系统应给出明确且友好的错误提示，指导用户重新操作。

4. UI/UX 设计特点：

1) 透明的信息呈现与决策辅助

支付页面将所有费用构成（商品费、配送费）透明化展示，避免用户在最后一步疑虑。关键金额数字使用加粗放大处理，强化视觉重心，辅助用户快速做出决策。

2) 流程化与防错设计

将支付流程分解为信息确认、方式选择、最终确认几个清晰步骤，符合用户的心理模型。通过清晰的视觉层次和未完成状态的提示，有效防止误操作，提升支付流程的顺畅度和成功率。

3) 安全感知与效率平衡

简洁专业的界面设计传递出安全可信赖感。默认支付方式建议或记忆上次选择，减少了用户的操作步骤。整个页面布局紧凑，确保了关键信息一目了然，实现了安全验证与支付效率的良好平衡。

(8) 我的订单页面设计

1. 整体布局

订单中心页面采用顶部选项卡与下方订单列表相结合的布局结构。顶部“订单中心”标题下方横向排列订单状态筛选选项卡，包括“全部”、“待支付”、“待接单”等，并以数字角标直观提示各状态下的订单数量。下方主体区域为按时间倒序排列的订单卡片流，每个订单卡片独立展示完整信息。底部保持应用标准导航栏，整体布局逻辑清晰，便于用户快速定位和管理不同状态的订单。

2. 核心信息展示设计

订单状态选项卡通过数字角标（如“全部(xx)”）提供总量概览，当前选中状态以强调色突出。每个订单卡片顶部显著标注订单号和当前状态（如“待支付”），商

家名称和订单总金额以突出字体显示。关键时间信息“下单时间”详细到秒，增强订单轨迹的精确性。根据订单状态动态呈现操作按钮组合，如“待支付”订单显示“取消订单”与“立即支付”，而“已完成”订单则无具体操作项，设计符合各状态下的实际业务逻辑。

3. 交互与反馈机制

用户点击不同状态选项卡时，订单列表即时刷新过滤，呈现对应状态的订单，并提供平滑的切换动效。点击订单卡片预期可展开查看详情。“立即支付”按钮直接链接至支付流程，“取消订单”操作需进行二次确认以防误触。页面支持下滑刷新以获取最新订单状态。

4. UI/UX设计特点：

1) 高效的状态管理与导航

通过顶部分类选项卡与数字角标，使用户对订单整体情况一目了然，并能一键直达特定状态订单集合，极大提升了订单管理的效率。视觉设计上明确区分了不同状态，降低了用户的认知负担。

2) 情景化的操作引导

根据订单的生命周期状态，动态呈现最相关、最核心的操作按钮，引导用户进行下一步合理操作。按钮设计主次分明，重要操作如“立即支付”采用强调色，次要操作如“取消订单”使用线框样式，有效防止误操作。

3) 信息层级清晰与可追溯性

订单卡片内部信息排版严谨，突出最关键的决策信息（商家、金额、状态），同时保留完整的辅助信息（订单号、精确时间）以供查证。整个页面设计确保了用户能够快速扫描、精准定位和轻松管理所有历史及当前订单。

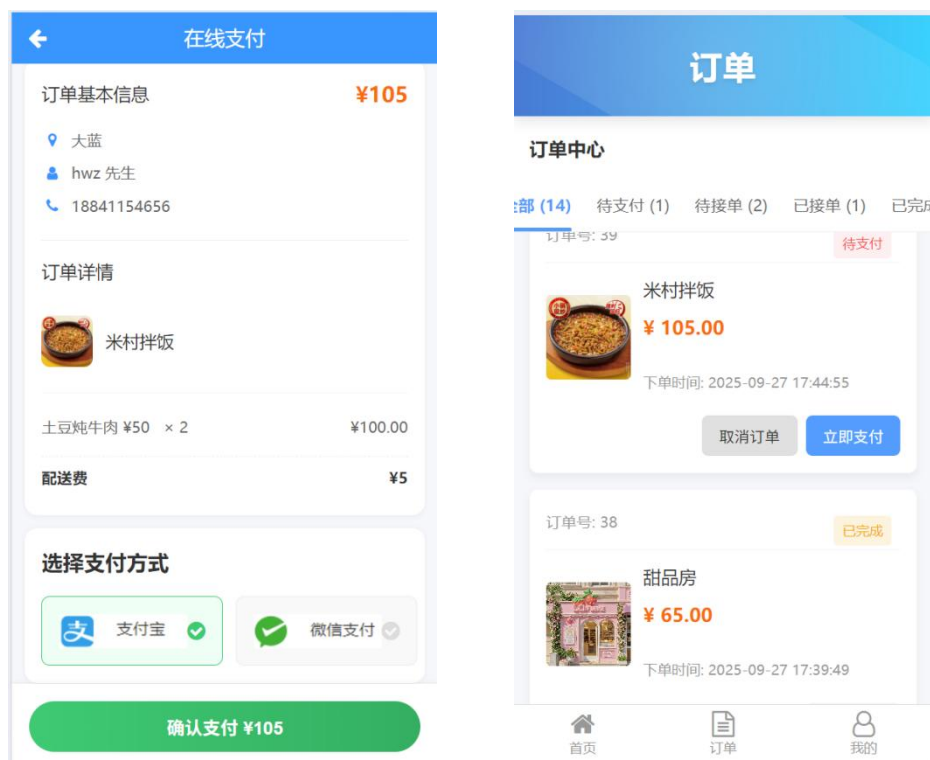


图2-5

(9) 个人信息页面设计

1. 整体布局

个人信息页面采用典型的列表式布局，以白色背景为基础，顶部突出显示用户核心身份信息，中部区域划分为账户操作与常用功能两大模块，底部保留标准导航栏。页面结构自上而下逻辑清晰，身份信息、账户管理、快捷功能依次排列，符合用户查阅个人资料的视觉流与操作习惯，整体风格保持简洁、高效。

2. 核心信息展示设计

页面顶部集中展示用户的核心身份标识，包括昵称、姓名、绑定的手机号码及电子邮箱，关键信息直接显露，无需二次点击。账户操作模块提供“切换到商家端”（若未注册商家，则显示“申请成为商家”）与“退出登录”两个重要入口，在视觉上与其他功能区分。常用功能区以网格或列表形式呈现“收货地址”、“我的收藏”、“消息与通知”等高频使用功能，图标与文字结合提升识别效率。

3. 交互与反馈机制

点击个人信息区域预期可进入编辑详情页面。功能入口触及时提供轻微的视觉反馈（如背景色变化）。“切换到商家端”操作前需进行身份验证确认，防止用户误触导致登录状态丢失。所有功能点击后均有明确的页面跳转或状态切换反馈。

4. UI/UX设计特点：

1) 信息直观与操作便捷

首要信息直接外露，减少用户获取核心信息的操作步骤。常用功能入口集中布

置，缩短用户访问路径。界面元素排版规整，视觉负担小，确保了信息查阅和功能操作的便捷性。

2) 安全性与风险控制

针对账户切换、退出登录等敏感操作，通过二次确认机制有效控制风险，避免误操作带来的不便。界面设计传达出稳定、可信赖的感受，保护用户账户安全。

3) 模块化与可扩展性

页面采用清晰的模块化设计，将不同性质的信息与功能进行合理分区，便于用户理解。这种结构也为未来新增功能或信息项预留了灵活的扩展空间，保持了界面的可持续性。

(10) 消息与通知界面

1. 整体布局

消息与通知页面采用垂直滚动列表布局，以时间倒序方式呈现所有系统消息。页面顶部居中显示"消息与通知"标题，下方为连续排列的消息条目卡片。每条通知占据独立区域，通过背景色和间距进行视觉区分。整体设计专注于信息的清晰阅读，没有复杂的操作控件，确保用户能够集中注意力于通知内容本身。

2. 核心信息展示设计

每条通知卡片采用完整的文本内容直接展示，如"恭喜！您的开店申请已通过审核，现在可以开始营业了"。不同状态的通知通过内容语义自然区分，审核通过的通知使用积极措辞，未通过审核的通知明确标注"抱歉"等提示性词语。消息文字采用标准字体大小，重要关键词未进行特殊样式处理，保持界面整体视觉统一性。

3. 交互与反馈机制

页面支持标准的下滑刷新操作，以获取最新的通知信息。点击单条通知预期可跳转至相关详情页面或对应业务模块（如店铺管理页面）。当前页面为纯信息展示界面，未设置批量操作或状态管理功能，保持操作逻辑的简洁性。

4. UI/UX设计特点：

1) 沉浸式的信息阅读体验

通过全文本展示和简洁的卡片布局，最大限度减少视觉干扰，为用户营造专注的阅读环境。通知内容直接明了，无需二次点击即可获取完整信息，提升信息获取效率。

2) 时序化的信息组织

严格按照时间倒序排列通知，确保用户优先看到最新消息。这种符合认知习惯的排列方式，帮助用户快速掌握最新动态，保证信息关注的时效性。

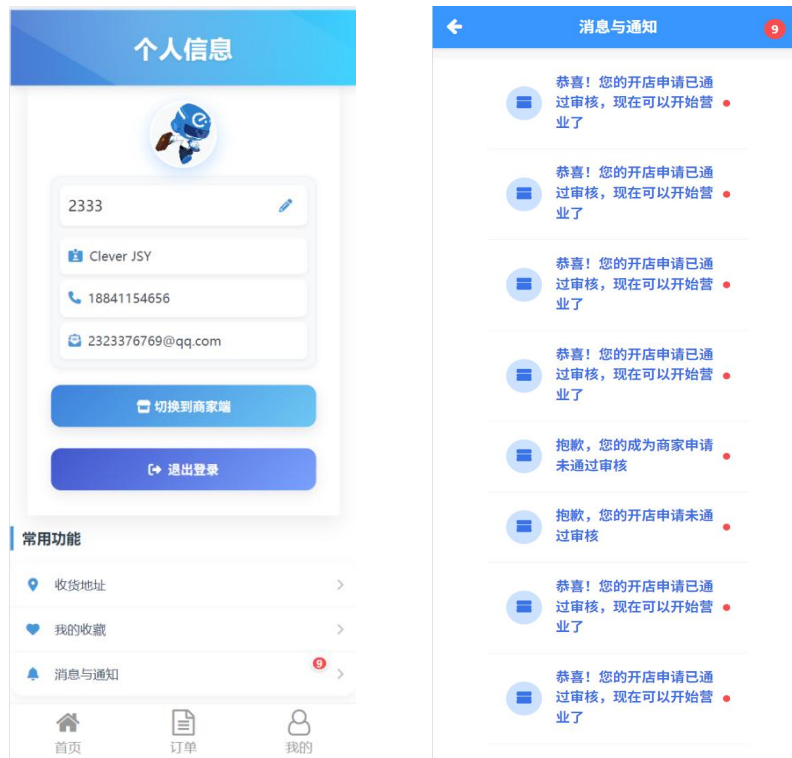


图2-6

2.3.2 商家用户

(1) 商家端个人中心页面设计

1. 整体布局

商家端个人中心页面采用功能导向型布局，以清晰的信息层级服务于商户的日常运营管理。页面顶部明确展示商家账号及脱敏处理的联系手机号，中部核心区域设置身份切换与账户安全操作入口，下部突出显示所管理的多个店铺及其关键互动指标。底部导航栏专为商家角色定制，包含“商铺”、“订单”、“我的”等核心业务入口。整体设计聚焦于效率与实用性，营造专业、稳重的操作氛围。

2. 核心信息展示设计

商家身份信息置于页面顶部，采用简洁的文本展示，确保账号识别的直观性。“切换为顾客”与“退出登录”按钮作为关键账户操作集中放置，在视觉上通过间距与色彩进行区分。店铺信息区域显著展示名称，并将“点赞”、“收藏”、“评分”三大核心社交证明指标并列呈现，以数据化形式直观反映店铺的受欢迎程度与口碑状况。

3. 交互与反馈机制

点击“切换为顾客”按钮将触发身份转换流程，并伴有状态切换提示，确保商户能快速回归顾客身份进行体验或验证。店铺互动数据（点赞、收藏、评分）区域支持点击查看详细数据趋势或用户评价列表。

4. UI/UX设计特点：

1)角色化与场景化设计

界面元素与功能入口紧密结合商家端的日常工作场景，弱化了普通消费者的娱乐化内容，强化了店铺管理、订单处理与数据分析等专业职能。实现了从消费者界面到管理者界面的无缝转换。

2)核心数据驱动决策

将反映店铺健康状况的关键指标（点赞、收藏、评分）置于页面显著位置，帮助商户快速评估店铺运营表现，体现了数据驱动运营的设计理念。设计为未来扩展更多经营分析工具预留了空间。

3)高效的身份管理与安全控制

提供便捷的身份切换通道，满足商户在不同角色间的灵活转换需求。同时对敏感操作施加必要的安全确认机制，在提升操作效率的同时确保了商家账户的安全性，平衡了便捷与安全两大诉求。

(2) 商家订单管理页面设计

1.整体布局

商家订单管理页面采用模块化垂直布局，以清晰的视觉层次展示多店铺订单处理流程。页面顶部设置店铺切换导航栏，下方依次为订单状态分类筛选区与详细订单列表。每个订单以独立卡片形式呈现，包含从订单状态、顾客信息到商品明细的完整数据链。底部固定商家专用导航栏，整体布局专注于提升订单处理的效率与准确性。

2.核心信息展示设计

店铺切换区突出显示商家下属店铺名称，方便商家快速切换管理视角。订单状态分类以选项卡形式展示各状态订单数量（如“全部(3)”），为商家提供全局业务概览。单个订单卡片内，订单号与状态标签置于顶部显眼位置，顾客联系信息（姓名、电话、地址）集中展示确保配送准确性。商品明细逐条列出并自动计算金额，最终汇总配送费与总金额，所有价格信息均采用右对齐方式增强数据可读性。

3.交互与反馈机制

商家可通过点击店铺名称实时切换不同店铺的订单视图。订单状态选项卡支持快速过滤不同状态的订单（订单在用户和商家端同时切换），点击后列表即时刷新。每个订单卡片预期支持展开/收起操作，便于快速浏览关键信息或查看完整详情。对于“待接单”等需操作的订单，设计预留了“接单”、“拒单”等操作按钮的位置，确保流程顺畅。

4.UI/UX设计特点：

1) 多维度订单筛选与监控

通过店铺与状态的双重筛选机制，使商家能精准定位到特定类型的订单，极大提升了订单管理的效率与针对性。关键状态（如待处理订单）配以数量提醒，帮助商家优先处理紧急任务。

2) 信息集中与流程化呈现

将订单生命周期中的所有关键信息（从顾客信息、商品内容到费用结算）结构化地集中展示，减少了商家在不同页面间切换的成本。信息排版遵循订单处理的实际逻辑流程，符合商家的操作认知。

3) 高效的数据识别与处理

采用表格化的数据对齐方式（如价格右对齐）、清晰的标签与分隔线，优化了信息的扫描速度，助力商家快速决策与操作。界面设计冷静、专业，避免了不必要的视觉干扰，专注于提升订单处理的核心效率。

(3) 商家商铺管理设计

1. 整体布局

我的商铺管理页面采用清晰的列表式布局，顶部设置店铺状态筛选选项卡，下方为主体内容区，依次展示各个店铺的简明信息卡片。页面以“全部”状态为默认视图，并提供“审核中”、“已上线”、“未通过”等状态筛选，方便商家快速分类管理名下所有店铺。底部固定商家专属导航栏，整体结构专注于提升多店铺管理的效率与清晰度。

2. 核心信息展示设计

每个店铺卡片独立成区，显著展示店铺名称，并集中列出关键经营参数：配送费、起送价以及商家地址。这些信息采用简洁的文本行式排版，确保核心决策数据一目了然。每个店铺卡片右侧固定位置配备“编辑”与“删除”操作按钮，形成统一的操作区域。页面最下方提供醒目的“申请新店”入口，激励商家扩展业务。

3. 交互与反馈机制

商家点击顶部不同状态选项卡时，下方店铺列表会即时刷新，只显示对应状态的店铺。点击“编辑”按钮将进入店铺信息修改页面，“删除”操作需触发二次确认弹窗以防止误删。“申请新店”入口引导至新店铺的注册与信息填写流程。页面支持下滑刷新，以同步店铺的最新状态。

4. UI/UX设计特点：

1) 集中化的多店铺管理

该页面实现了对商家名下所有店铺信息的集中展示与统一管理，通过状态筛选机制，商家可以迅速掌握各店铺的运营状况（如上架、审核中），极大提升了管理多个店铺的便利性。

2) 高效的信息扫描与操作

店铺卡片的布局高度标准化，关键经营参数（费用、地址）前置显示，支持商家快速扫描和比较不同店铺的设置。将“编辑”、“删除”等常用操作外露并固定位置，缩短了操作路径，提升了管理效率。

3) 清晰的业务状态与扩展引导

通过视觉设计清晰区分不同审核状态的店铺，帮助商家明确后续操作重点。

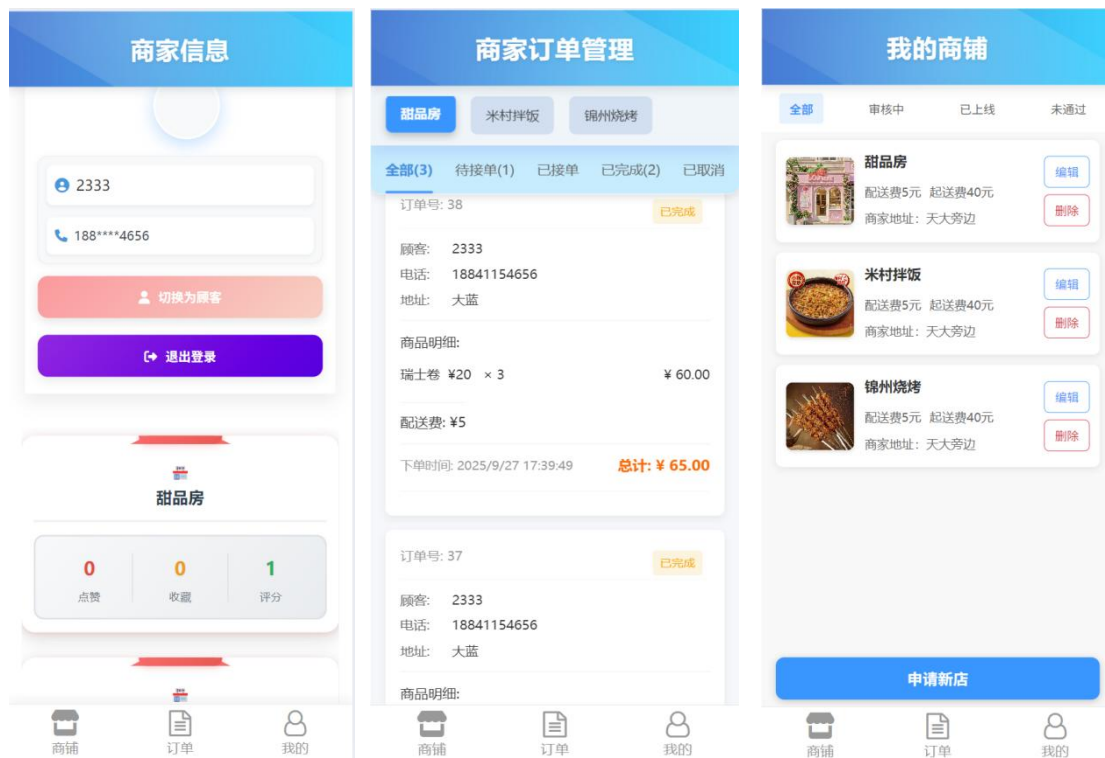


图2-7

2.3.3 管理员用户

(1) 管理员首页页面设计

1. 整体布局

管理员管理平台页面采用数据看板式布局，以清晰的模块划分呈现系统核心数据与管理入口。页面顶部左侧展示当前管理员身份信息，右侧设置安全退出口。主体区域上部为关键数据指标总览面板，下部为待办审核任务区。底部提供核心功能导航栏，整体布局注重信息密度与操作效率，体现管理平台的权威性与专业性。

2. 核心信息展示设计

管理员身份区分明确显示用户名及其唯一标识ID，采用复选框样式增强选中状态感知。数据总览面板使用三列表格形式突出显示"总用户人数"、"总店铺数"和"总营业额"三大核心指标，关键数字采用加粗放大字体强化视觉冲击。审核任务区通过"用户审核（数量）"和"商铺审核（数量）"标签直观提示待处理任务量，无待办任务时显示友好提示"暂无待审核的商家申请"。

3. 交互与反馈机制

点击数据总览面板的任一指标预期可跳转至对应详细数据页面。审核任务标签支持直接点击进入相应的审核管理界面。退出登录操作需进行二次确认以确保系统安全。底部导航栏高亮当前所在模块，提供"首页"、"用户管理"、"商铺管理"等核心功能的快速切换。

4. UI/UX设计特点：

1)数据驱动的决策支持

通过突出显示关键业务指标，为管理员提供全局业务洞察，支持数据化决策。指标设计聚焦核心KPI，布局紧凑且信息量大，便于快速掌握平台运营状况。

2)任务导向的高效处理

将待办审核任务置于显著位置，并通过数字角标实时提醒，确保重要任务得到及时处理。界面设计简化了管理操作路径，提升平台管理效率。

3)安全与权限控制

通过明确的身份标识和安全的退出机制，强化平台安全管理。功能模块的权限划分清晰，导航结构简单直观，既保证了操作的便捷性，又确保了系统的安全性。

(2) 用户管理页面设计

1. 整体布局

用户管理页面采用功能优先的列表式布局，顶部设置全局搜索栏与用户状态筛选选项卡，下方为主体内容区展示用户列表。页面以清晰的视觉层次区分搜索、筛选和数据显示区域，底部保留管理平台导航栏。整体设计注重信息检索与批量操作的效率，满足管理员对用户账户的日常管理需求。

2.核心信息展示设计

搜索栏预设提示语"搜索用户名、手机号或邮箱"，支持多关键词模糊匹配。状态筛选选项卡提供"全部用户"、"已启用"、"已禁用"三种过滤条件，当前选中状态高亮显示。用户列表按状态分组展示，"已启用"与"已禁用"分区明确，每个用户条目包含用户名、手机号、邮箱和注册时间等完整身份信息，关键操作按钮"禁用"/"启用"置于条目右侧固定位置。

3.交互与反馈机制

搜索框支持实时输入联想，输入过程中动态显示匹配结果。状态选项卡点击即时刷新列表，筛选结果即时呈现。操作按钮设有悬停状态变化，点击"禁用"/"启用"时触发二次确认对话框，防止误操作。列表支持分页加载，确保大量数据下的浏览流畅性。

4.UI/UX设计特点：

1) 高效检索与状态管理

通过组合式搜索条件（用户名、手机号、邮箱）和状态筛选机制，使管理员能够快速定位目标用户账户。清晰的视觉分组和状态标识，让用户账户状态一目了然，大幅提升管理效率。

2) 完整的信息透明度

每个用户账户展示完整的联系信息和注册时间，为管理员决策提供充分依据。关键操作外露且位置固定，保证操作的一致性和可预测性，降低学习成本。

3) 安全的操作控制

针对账户状态变更等敏感操作，设置必要的确认流程，确保管理操作的谨慎性。界面保持冷静、专业的视觉风格，避免不必要的干扰，专注于用户管理这一核心任务。

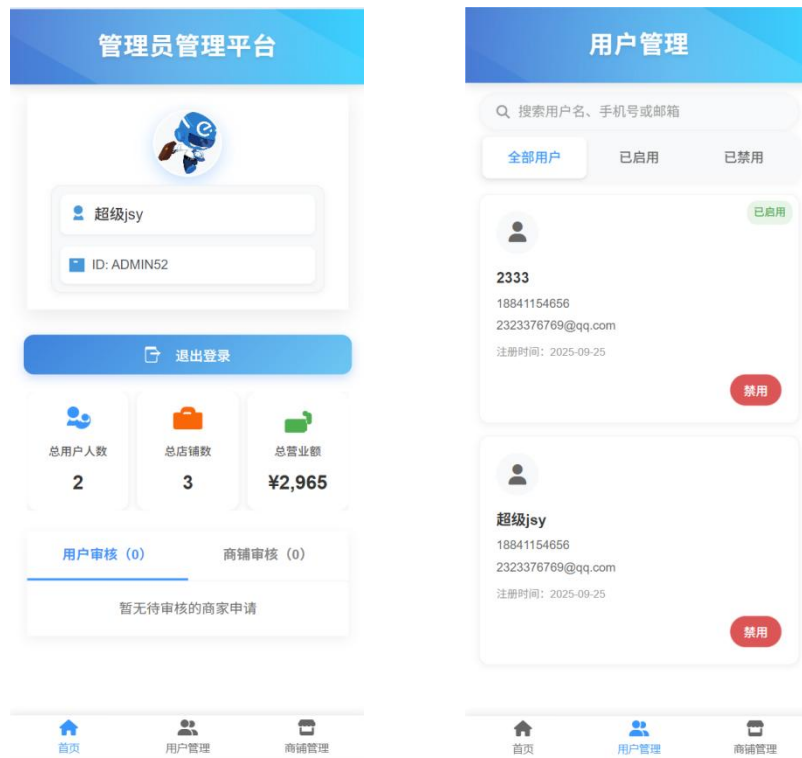


图2-8

(3) 商铺管理页面设计

1. 整体布局

商铺管理页面采用简洁明了的信息卡片式布局，顶部突出显示商家集团信息，下方提供核心操作入口。页面以白色为背景主色调，关键信息区域通过适当的留白和分割线进行视觉区分。底部固定管理平台导航栏，且"商铺管理"选项处于高亮激活状态，明确指示当前所在页面位置。整体设计风格保持高度的专业性和清晰度，便于管理员快速识别商家主体并进行后续操作。

2. 核心信息展示设计

页面顶部显著位置展示商家集团名称"xxx集团"，下方明确标注其主要联系方式，确保管理员能够快速获取关键联系信息。核心操作区域设置醒目的"查看商铺"按钮，采用高对比度色彩设计，引导管理员进一步查看该集团旗下的详细店铺列表。所有信息元素排版紧凑且逻辑分明，减少了不必要的视觉干扰。

2. 交互与反馈机制

点击"查看商铺"按钮将直接跳转至该集团旗下的具体店铺管理页面，实现集团层面到店铺层面的无缝衔接。页面加载过程配有适当的过渡动画，确保操作反馈的及时性。底部导航栏提供在不同管理模块间的快速切换能力，提升平台整体操作效率。

3.UI/UX设计特点:

1) 层级化的信息架构

通过从集团信息概览到具体店铺查看的递进式设计，构建了清晰的信息层级架构。这种设计既满足了管理员对商家集团层面信息的基本了解需求，又为深入查看具体店铺详情提供了便捷入口。

2) 高效的核心操作引导

将最常用的"查看商铺"操作以主要按钮形式突出展示，显著缩短了管理员的决策路径。界面元素布局极简，避免了冗余信息的干扰，确保管理员能够快速完成核心管理任务。



图2-9

2.4 安全设计

2.4.1 密码安全策略

系统采用多层次安全措施保护用户密码等敏感数据，确保数据在传输和存储过程中的安全性。

(1)BCryptPasswordEncoder加密存储

系统采用Spring Security提供的BCryptPasswordEncoder进行密码加密存储。

(2)盐值加密机制

BCrypt自动生成随机盐值(salt)并与密码组合，相同密码每次加密都会产生不同的哈希值，能有效防止彩虹表攻击和字典攻击。

2.4.2 身份认证机制

系统采用Spring Security + JWT组合方案实现身份认证与授权管理。

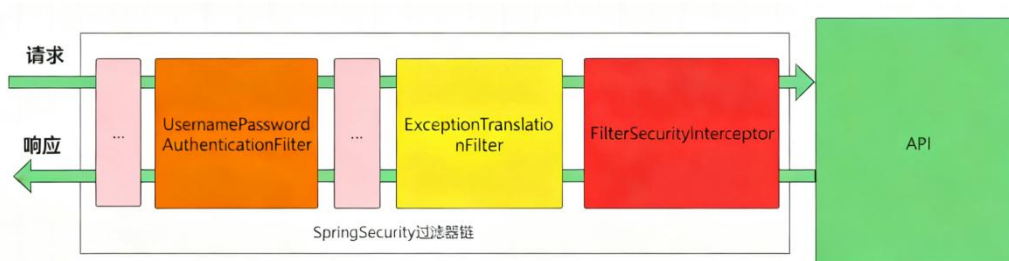


图2-11 身份认证流程

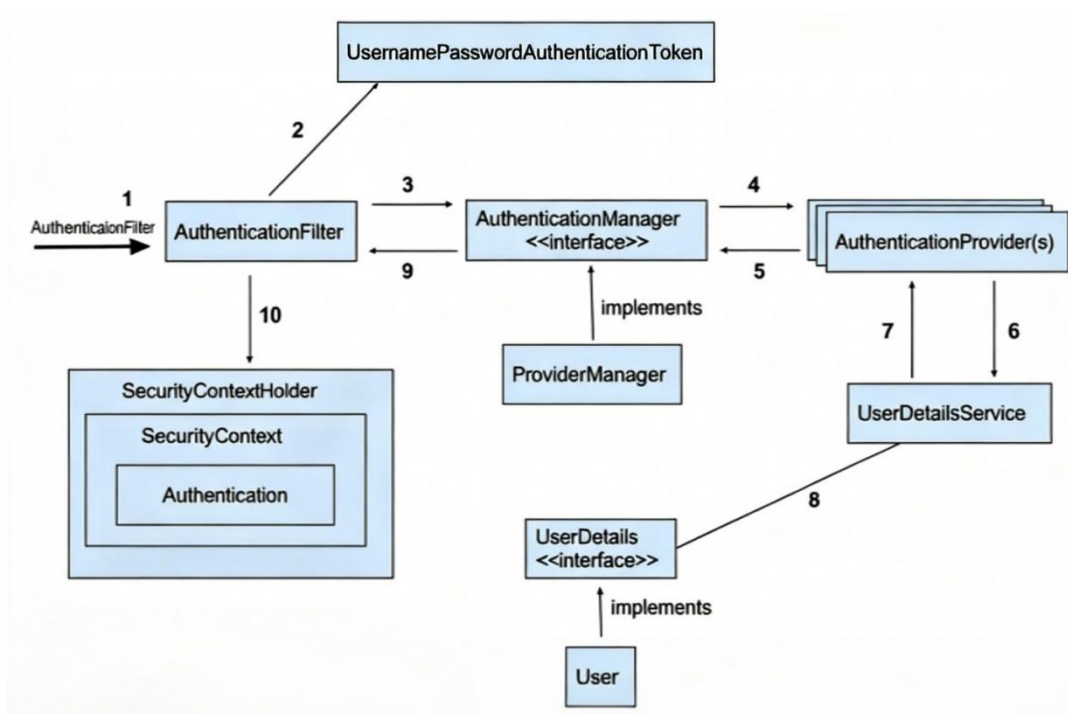


图2-12 身份认证机制

(1) 自定义登录流程

1. 认证处理：

- 自定义登录接口调用ProviderManager进行用户认证
- 认证通过后生成JWT令牌并返回客户端

2. 用户查询：

- 实现UserDetailsService接口查询数据库用户信息
- 返回UserDetails对象供Spring Security认证使用

(2) JWT认证过滤器

1.Token处理流程:

- 从Authorization请求头中提取JWT令牌
- 解析Token获取用户标识信息

2.安全上下文管理:

将用户认证信息存入SecurityContextHolder, 为后续请求处理提供身份验证状态, 以确保控制器方法能够获取当前用户信息

(3) 前后端安全协作

1. 前端: 在创建好的axios实例 axiosInstance 上, 挂载两个拦截器

1) 请求拦截器:

- 前端向后端发送请求之前拦截;
- 在 header添加 token, 若有请求错误, 在控制台打出

2) 响应拦截器:

- 后端向前端发送响应, 前端接到之后拦截;
- 处理响应状态码错误 401 , 500 等
- 网络错误, 重试三次, 重试失败, 跳转回主页;
- 重试次数超过最大限制 3, 跳转回主页

2.后端:

1)所有请求在进入Controller层之前均经过安全过滤器链

2)JWT认证过滤器优先进行Token验证和用户身份解析

3)登录成功生成JWT令牌并返回前端

4)后续所有请求必须在Authorization头中携带有效Token, 无效或过期的Token请求将被拦截

Token认证与拦截器的结合为系统提供了一种高效、安全且用户体验良好的认证方式, 既保证了API的安全性, 又确保了系统的可扩展性和维护性。

2.5 细节设计

2.5.1 公用返回组件设计

饿了么1.0中,页面并未添加返回组件,实现页面的返回功能只能依靠浏览器的返回,于是饿了么2.0中设计了返回组件作为公用组件,在需要返回功能的页面中导入该组件。

2.5.2 显示设计

饿了么 V1.0中,订单以及购物车的小数显示上存在小数显示的 bug,饿了么 V2.0在前端相关位置添加小数显示限定函数进行显示限制。

饿了么 V1.0中,前端版本代码均存在商家列表/商品列表以及首页不能拉到底,最后一个信息会被底部菜单栏遮挡的问题,饿了么 V2.0通过在底部添加一占位白块解决。

饿了么V1.0中对于用户的反馈显示使用的是浏览器默认的弹窗提示,用户体验差,饿了么V2.0将设计专门的提示组件,并对整个项目的前端提示做出重构,提升用户体验。

2.5.3 历史订单修改逻辑设计

在对订单中涉及的商品名称、价格以及送货地址等信息进行修改时,会引起历史订单信息的修改.可以通过修改数据库结构,为商品和地址添加删除标记,在删除和修改商品信息或者地址信息时,并非真正的删除和修改,而是将原有数据的删除 tag 置为 1,原数据依旧保留在数据库中,订单读取数据时根据该条数据的唯一 id 进行查找,依旧是原本的数据。

2.5.4 手机号验证设计

在登录/注册以及地址信息的填写中,需要保证手机号格式正确,饿了么将在前后端进行手机号规范验证,具体内容为:

- 手机号不能为空
- 手机号位数为11位
- 手机号要以1开头
- 手机号第二位数字大于2

上述验证不仅仅在前端实现,在后端同样需要进行二次验证,并且根据不同的错误返回不同的错误码,正确则由后端返回 1,手机号为空返回-1,长度错误返回-2,首位不为 1 返回-3,格式不正确返回-4。

2.5.5 部分逻辑优化设计

饿了么V1.0中,后端代码订单业务逻辑缺少对订单和购物车是否为空的验证.在生成购物车时添加对购物车以及订单是否为空的逻辑验证。

第三章 人员分工

高灿

登录注册、鉴权部分（SpringSecurity+JWT），用户管理（启用与禁用），顾客申请成为商家，商家申请开店，管理员审核申请，管理端数据看板，消息与通知模块的后端实现，用WebSocket实现顾客端与商家端订单状态的实时切换、顾客或商家发送申请与管理员审核申请的实时消息推送。以及上述部分的前后端联调。前端高德api的接入，图片OSS上传。

曾意

主要负责商铺商家功能模块的后端架构与开发，完整实现了商家信息管理、业务流转等核心逻辑。同时，完成用户互动链路中点赞、收藏功能的实现。此外，完成了第三方智能体服务的后端集成。在项目部署阶段，负责服务上云与运维配置工作。在上述各功能开发过程中，深度参与前后端协作联调，通过编写规范的接口文档、及时定位并协同解决技术问题，有力支撑了功能从接口设计到前端呈现的全流程顺利交付。

杨晓越

主要负责顾客端与商家端共19个页面的前端开发与样式实现，并配合团队完成了“我的”页面在双端视角下的功能联调与数据对接工作。在团队协作中，我协助实现了用户身份切换功能，处理了相关的页面渲染与权限更新，并在这个过程中为前端界面风格的统一做了一些推进性工作，有效提升了前端整体的视觉规范与专业度。

贾思韵

完成了包括18个前端页面的设计与静态代码开发，并负责顾客首页的前后端联调工作，确保页面功能与数据交互的准确实现；同时，也独立完成了AI智能体部分的前端集成与界面交互开发，保障了智能体服务在用户端的顺畅使用。

戴茗静

主要负责了项目全局异常处理部分的封装；完成商品、购物车、订单部分的接口编写，整个下单流程和商家商品管理功能链路的后端实现，以及上述功能的前后端联调。还参与测试，排查问题，修复bug的流程。

第四章 项目开发过程

为了方便项目的开发进度掌握，我们将整个项目开发周期的三周划分为六个阶段，在每个阶段都制定了详细的小组成员分工、本阶段应该完成的工作和后续待完成的工作。具体阶段的安排为：

- 第一阶段：9.8-9.13
- 第二阶段：9.14-9.18
- 第三阶段：9.19-9.20
- 第四阶段：9.21-9.23
- 第五阶段：9.24-9.25
- 第六阶段：9.26-9.27

4.1 第一阶段（9.8-9.13）

第一阶段为需求分析阶段，该阶段的我们的核心任务是进行需求分析，明确要实现哪些功能，明确项目的目标、范围、功能列表、用户故事及性能要求等，并撰写详尽的项目需求文档，为后续所有开发工作奠定坚实的基础。同时组内成员商量前后端分工，去学习对应的开发知识。

4.1.1 第一次组会记录（9.8）

- (1) 下载并仔细阅读了老师提供的饿了么V1.0工程代码
- (2) 组员在各自的电脑上装配好前后端环境，将已有的基础代码正常跑起来
- (3) 对于会前提出的13条项目改进点进行逐条讨论，确定每一条是否能够实现
- (4) 共同学习智慧树上的学习资料，讨论并确认《软件需求规格说明书》具体要写什么内容、要画哪些UML图
- (5) 小组成员对《软件需求规格说明书》的撰写进行分工：
 - 曾意负责业务分析部分的类图绘制，包括订单状态业务图、用户基本信息业务图、商户铺管理图和商品管理业务图、用户管理业务图和点赞收藏评分业务图。
 - 贾思韵负责引言部分的项目简介、背景、开发路线、项目现状、目前工作的分析，需求分析、页面设计。
 - 杨晓越负责项目设计部分数据库设计的整理及需求分析部分内容，包括现有数据库表单的统计和部分页面设计文档补充。

- 高灿负责功能性需求中UML用例图的绘制与相关表格的撰写，包括总用例、用户的用例、顾客的用例、商家的用例、管理员的用例，以及非功能性需求中的安全性分析。

- 戴茗静负责UML活动图的绘制，将整个项目梳理成三个大的功能链路模块：用户登录注册与管理、订单处理、商家商铺管理；并初步汇总了项目功能需要的后端可能接口

4.1.2 第二次组会记录 (9.10)

(1) 汇报这两天的学习进度与文档撰写进度

(2) 针对《软件需求规格说明书》的细节部分进行讨论

(3) 对于前后端代码撰写进行了初步分工：戴茗静、曾意、高灿主后端，贾思韵、杨晓越主前端

(4) 于9.11完成了《软件需求规格说明书》的初版撰写。

4.2 第二阶段 (9.14-9.18)

第二阶段是系统基础设计与开发阶段，在此阶段，组内并行开展前端页面的原型设计、视觉风格定稿以及用户交互流程设计，同时后端完成系统的核心框架搭建与基础数据接口的开发工作，确保系统主体结构稳定。

9.14晚上老师上传了饿了么V2.0的代码。

4.2.1 第三次组会 (9.15)

(1) 组内成员在会前仔细阅读了饿了么V2.0代码，明确了老师规定的接口文档中的基础功能

(2) 结合老师的要求，我们对之前讨论的新增需求进行了修改，明确了要增加管理端，并初步商讨了管理端要做哪些功能：数据看板、审核申请、管理用户、管理店铺

(3) 提出新增WebSocket用于实现实时消息推送

(4) 对于将要完成的工作进行明确分工：在下次组会之前前端先撰写缺少的所有静态页面，后端先将老师规定的接口文档中的所有接口开发出来

(5) 讨论了班级开进度汇报会时需要汇报哪些内容，PPT制作分工，杨晓越同学负责了第一次的汇报

4.3 第三阶段 (9.19–9.20)

第三阶段聚焦于功能模块的具体实现，开发工作将围绕新增的特定功能展开，包括设计与实现这些功能对应的后端业务逻辑接口，并编写完成前端静态页面、组件及用户交互逻辑代码，使各项功能在前后台均得到初步实现。

4.3.1 第四次组会记录 (9.19)

(1) 组内成员在组会开始时先进行了代码合并，并且集中解决合并冲突问题，对模糊问题进行二次商讨和决定

(2) 基于前端新开发的页面，以及后端实现的所有接口，展开细节的讨论，围绕新增的功能逐步确定了后续需要继续改进和优化的细节，如浮点数精度问题、店铺相关代码的权限、商家端剩余页面的新功能UI

(3) 分工：

- 后端将开发商铺、商品、订单及用户管理四大核心模块，包括商家申请与审批、商品上下架、订单状态流转、购物车管理等功能接口，并完善权限控制与数据统计。
- 前端同步进行20个新页面的UI/UX开发与优化，以匹配新增的商家管理、收藏点赞、订单流程及用户交互等业务需求。
- 由曾意同学负责第二次的汇报展示

4.4 第四阶段 (9.21–9.23)

第四阶段进入前后端联调环节，本阶段旨在将分别开发的前端页面与后端服务进行集成对接，通过细致的接口联调与数据互通测试，验证系统各模块间的协调性与功能完整性，确保数据传输准确、业务流畅通。

4.4.1 第五次组会记录 (9.21)

(1) 确定前后端联调的细节，组内成员一起进行接口的联调与数据互通测试。其中后端的三位同学承担了大部分联调工作，前端的同学补充了页面样式统一、导航栏设计等部分代码

(2) 接口联调的分工

- 高灿：用户地址管理、管理员首页与用户管理、登录通知、注册页面等核心模块
- 曾意：商铺信息展示与列表、收藏功能、商家视图等核心模块
- 戴茗静：商家信息详情、购物车、订单流程、商品提交等核心交易模块
- 贾思韵：顾客首页、我的信息页面等模块
- 杨晓越：商家资料、支付成功页等模块

(3) 对第三次汇报内容进行讨论，曾意同学负责了PPT的制作与汇报

4.4.2 第六次组会记录 (9.23)

- (1) 完成全部前后端联调工作，成功整合了由各成员独立开发的功能模块，实现了系统整体的统一
- (2) 各功能页面与模块间的路由跳转已全部打通，确保了用户从浏览、操作到支付的完整流程顺畅无阻
- (3) 项目正式进入全面测试阶段，将采用涵盖正常与边界场景的多样化测试用例，系统验证各功能的稳定性与数据处理的准确性

4.5 第五阶段 (9.24-9.25)

第五阶段是全面的测试与优化阶段，将对整个项目进行系统性的功能测试、性能测试及兼容性测试，并集中定位与修复测试过程中暴露的所有缺陷与漏洞，通过多轮迭代确保项目交付质量达到预定标准。

4.2.1 第一轮全面测试

- (1) 首先进行第一轮全面问题排查，继第四阶段各模块互通，小组每人再对自己负责的部分进行测试，寻找不合理点与优化点进行再次改进，并协商存在争议的部分
- (2) 检查页面样式的疏漏，发现在git分支合并时丢失了部分修改代码，于是统一排查所有页面的布局并将其恢复正常；同时改进页面细节，如小数显示、位置偏差等
- (3) 测试后端基础接口，增强其在各种异常情况下的鲁棒性；对于前后端交互，在整个项目的视角上进行不同异常状况的全面测试并改进

4.2.2 第七次会议 (9.25)

- (1) 为迎接即将到来的最终展示，小组成员共同在现场进行第二轮的全面测试，确保整个项目所有功能的正确，并充分准备最终展示。
- (2) 每个人都对整个项目所有功能进行完整测试，将找出的问题列举在共享文档上。
- (3) 针对特殊模块，解决耗时较长、业务bug调整、页面样式细节问题进行分工，进行低耦合的问题解决，最终高效且低遗漏的合并解决后的代码，最终实现整个项目的无差错。
- (4) 最终展示的流程准备，主要包括：数据准备、展示文稿、展示流程准备，并进行了模拟展示。同时为确保万无一失，每位组员都熟悉整个流程，并在电脑上准备展示数据。
- (5) 撰写项目特别功能清单，分为详细版和概括版
- (6) 高灿同学负责了最终的验收展示

4.6 第六阶段 (9.26-9.27)

第六阶段为项目收尾与总结阶段，此阶段的主要工作是梳理项目全过程，汇总开发数据、测试结果及管理经验，并撰写详实的项目总结报告，对项目成果、技术难点与过程反思进行系统化归档，为未来工作提供参考。

4.6.1 完成最终展示 (9.26)

结束了最终的展示，对老师的点评与建议进行总结，记录待完善的改进点

4.6.2 第八次会议 (9.26)

(1) 商讨课程最终版项目代码的细微改进点，进行第三轮的全面测试，总结项目功能流程以及此次实践的经验。

(2) 确定撰写实践报告的分工，总体上按各自前面负责的部分，其中联调测试、人员分工、项目开发过程、问题与解决、结果反思等由所有组员共同撰写。其他部分分工具体如下：

- 曾意：部分后端接口设计；部署文档；问题、解决与反思部分内容
- 贾思韵：前端测试文档、部分项目特色说明文档
- 高灿：部分后端接口设计、部分后端测试文档与前后端联合测试
- 戴茗静：数据库设计，部分后端接口设计、部分后端测试文档
- 杨晓越：前端测试文档、部分项目特色说明文档

4.6.3 文档、报告、总结等后续工作

(1) 尝试将项目在本地与云服务器上的部署，撰写详细的部署文档，并为项目撰写详细的 README 介绍。

(2) 整理git分支，确认最终版代码的版本

(3) 按分工撰写实践报告

第五章 项目测试文档

5.1 测试概括

5.1.1 项目背景.

饿了么是一款面向各年龄段用户的外卖服务平台，旨在让用户足不出户即可便捷地享受多种菜系的美食。项目通过提供清晰的商家信息与菜品展示，优化用户从浏览到下单的完整体验。

5.1.2 编写目的

本文档旨在明确“饿了么”项目前端部分的测试策略、范围、方法及结果。其主要目标读者为软件开发团队与质量审核人员。通过阐述测试流程与发现，旨在为功能审核提供依据，并确保开发团队对前端质量有统一认知。

5.2 前端测试

测试人员：前端测试的主要执行人员为：贾思韵、杨晓越。

5.2.1 测试目标

本次前端测试的核心目标如下：

1. 功能正确性：验证所有前端交互功能均按照需求规格正常实现
 2. 用户体验：确保界面美观、操作流畅、符合设计规范
 3. 兼容性与响应式：保证应用在主流的浏览器、操作系统和设备上表现一致
- 前端性能：保障页面加载速度与交互响应时间满足用户体验要求

5.2.2 测试范围

(1) 本次测试将覆盖以下前端模块/功能：

1. 用户认证模块：用户登录、注册
2. 主数据展示页面：商家列表、收藏列表、管理端数据展示页面
3. 核心交互流程：商品搜索、筛选、加入购物车、下单
4. 用户个人中心：信息修改、头像上传、收货地址管理
5. 管理后台功能：数据的增、删、改操作

(2) 排除范围：

1. 后端API接口的内部逻辑（由后端单元测试覆盖）

2. 第三方服务（如支付、短信）的端到端连通性测试

5.2.3 测试方法

本次测试综合运用了多种测试方：

- (1) 系统的功能测试选用了手工测试，运用黑盒测试中的等价类划分、边界值分析、错误推断、因果图法。
- (2) UI方面的测试包括：易用性测试、规范性测试、帮助设施测试、合理性测试、美观与协调性测试、独特性测试、快捷方法组合组合测试。

5.2.4 测试执行

(1) 功能测试

方法：黑盒测试，综合运用等价类划分、边界值分析、错误推断法。

测试重点：

- 表单交互：输入校验、提交、重置、错误提示
- 页面导航：路由跳转正确性、面包屑、链接有效性
- 动态功能：数据加载、排序、筛选、分页、模态框等
- 业务逻辑：前端控制的业务规则

(2) UI/UX 测试

方法：手工测试、视觉对比。

测试重点：

- 视觉一致性：与UI设计稿（原型图）在布局、颜色等方面保持一致
- 交互反馈：鼠标悬停、点击、禁用等状态的样式变化
- 易用性：界面布局合理，信息层级清晰，用户操作路径明确

(3) 兼容性测试

测试方法：手工测试

测试重点：

- 跨浏览器测试：在不同品牌和版本的浏览器上验证功能和样式
- 响应式布局：在不同屏幕尺寸和设备上，布局是否正常，功能是否可用

5.2.5 测试内容

(1) 功能测试

验证系统功能是否符合其需求规格说明书，核实系统功能上是否完整，没有冗余和遗漏功能。详细介绍如表5-1。

表 5-1 功能测试介绍

说明	描述
测试范围	验证数据精确度、数据类型、业务功能等相关方面的正确性
测试目标	核实所有功能均已正常实现、即是否与需求一致
技术	黑盒测试（边界值分析、等价类划分）
工具与方法	手工测试
开始标准	开发阶段对应的功能完成并且测试用例设计完成
完成标准	测试用例全部执行通过，高优先级缺陷已解决

具体测试用例示例：

1. 点击“注册”按钮，成功跳转至注册页面
2. 提交表单时，前端正确校验手机号格式与密码格式
3. 添加商品至购物车后，购物车图标数量实时更新
4. 页面跳转、按钮交互、动态内容加载（如菜品列表）等功能正常

(2) 用户界面测试

测试用户界面是否具有导航性、美观性、规范性、是否满足设计中客户要求的执行功能、详细介绍如表5-2 UI测试。

表 5-2 UI测试介绍

说明	描述
测试范围	保证窗口运行、提示信息、页面跳转、易于使用等方面的正确性
测试目标	核实各个窗口的风格（包括颜色、字体、提示信息、图标、title等）均与需求保持一致
技术	Web 测试通用方法
工具与方法	手工测试、目测
开始标准	界面开发完成
完成标准	UI 符合可接受标准，能保证用户界面的友好性，易操作性，而且符合用户操作习惯

测试重点：

1. 易用性：界面布局合理，用户能轻松找到所需功能
2. 规范性：同类按钮、提示信息风格统一

3.美观与协调性：字体、颜色、间距等符合设计稿

4.响应式布局：在不同屏幕尺寸下布局正常（与兼容性测试协同）

(3) 回归测试

说明：在代码修改后，验证原有功能未被破坏。

表 5-3 回归测试介绍

说明	描述
测试范围	所有功能、用户界面、兼容性等测试类型
测试目标	核实代码修改后，现有功能、性能等均未引入新的缺陷
技术	黑盒测试
工具与方法	手工测试（针对关键路径）
开始标准	被测试的软件或其环境发生变更时
完成标准	核心功能测试用例全部通过

近期回归测试案例：

- 1、修复“注册时头像上传失败”问题后，验证整个注册流程及头像上传功能
- 2、修改“首页搜索框提示文字”后，验证首页显示及搜索功能正常
- 3、更新“重复申请商家权限的提示信息”后，验证提示能正确显示后端返回的 Message

5.2.6 测试结果

- (1) 缺陷修复情况：测试过程中发现的主要问题，如“AI咨询功能前端无返回结果”、“修改头像数据传递不稳定”等，均已由开发团队修复。
- (2) 回归测试结果：针对上述修复进行的回归测试结果通过，确认问题已解决且未引入新的缺陷。
- (3) 总体评价：当前版本的前端功能实现完整，业务流程畅通，UI界面符合设计规范，兼容性与性能满足基本要求，已达到可发布标准。

5.3 后端测试

测试人员：本次后端测试的主要执行人员为：戴茗静、曾意、高灿。

5.3.1 测试内容

(1) 利用Apifox进行接口测试

1. 执行方式:

- 1)单个接口测试: 针对新增 / 修改接口, 先单独执行验证基础功能
- 2)接口场景测试: 按 “用户下单→商家接单→顾客查订单” 等业务流程, 在 Apifox 中创建 “场景用例”, 批量执行并校验流程完整性
- 3)WebSocket 联动测试: 执行 “顾客下单” 接口后, 通过 Apifox 的 “WebSocket 调试” 功能, 监听对应商家的连接, 验证推送消息是否实时且正确。

2. 结果记录

- 1) 正常用例: 标记 “通过”, 记录响应耗时 (要求核心接口 $\leq 500\text{ms}$)
- 2) 异常用例: 标记 “失败”, 截图保存响应信息、数据库数据, 并标注 “接口名称 + 用例 ID + 异常描述”
- 3) 遗留问题: 若存在 “非阻塞性问题” (如响应耗时略超阈值), 记录为 “待优化”, 跟踪后续版本修复。

3. 测试完成标准

- 1) 覆盖接口文档中 100% 的接口, 无遗漏核心业务接口
- 2) 每个接口的正向用例、关键异常用例 (参数缺省、权限错误、数据非法) 通过率 100%
- 3) WebSocket 推送功能验证通过: 顾客下单后, 商家端 1 秒内接收新订单消息, 无延迟或丢失
- 4) 接口响应耗时: 核心接口 (下单、接单、查询) 平均耗时 $\leq 500\text{ms}$, 峰值 $\leq 1000\text{ms}$
- 5) 输出《Apifox 接口测试报告》, 包含用例执行统计、缺陷清单、优化建议。

(2) 基于 Mybatis-Springboot-starter-test 的后端单元测试

1. 测试依赖配置

在项目pom.xml中引入 Mybatis-Springboot-starter-test 及相关配套依赖, 确保单元测试可直接操作 MyBatis 映射接口、集成 Spring 容器环境, 同时避免污染生产数据库。

2. 测试环境准备

- 1) H2 内存数据库配置: 在 src/test/resources 下创建 application-test.yml, 配置测试环境数据库 (使用 H2 内存库, 每次测试后自动清空, 避免数据残留)
- 2) 测试 SQL 文件准备: 在 src/test/resources/sql 下创建 schema-test.sql (订单表结构, 和 MySQL 订单表结构相同) 和 data-test.sql (测试数据)

3. 测试范围与核心测试对象

本次单元测试聚焦MyBatis 数据访问层 (Mapper 接口) 和业务逻辑层 (Service), 覆盖项目核心业务模块, 部分测试对象如表5-3。

表 5-3 核心对象测试

测试层级	测试对象	核心测试场景
Mapper 层	BusinessMapper (商家映射接口)	1.按 ID 查商家信息 (getBusinessById/selectBusinessById) 2. 按用户 ID 查商家列表(selectByUserIdAndStatus) 3. 新增商家(insertBusiness/applyForAddBusiness) 逻辑删除商家(deleteBusiness) 4.搜索商家(searchBusinesses/searchByKeyword)
Service 层	BusinessService (商家业务逻辑)	1.商家信息更新(updateBusiness/patchBusiness, 含权限校验) 2.商家删除(deleteBusiness, 含所属权校验) 3.申请新增商家(applyForAddBusiness, 区分商家 / 管理员权限) 4.商家搜索排序(getBusinessesBySearch, 按评分 / 销量排序) 5. 轮播商家查询(getBusinessesInCarousel, 取 Top3 高评分高销量)

4. 执行方式

- 1) 单个测试类 / 方法：在 IDE 中右键点击测试类 / 方法，选择 “Run Test”，直接执行并查看结果
- 2) 批量执行：通过 Maven 命令 `mvn test` 执行所有测试用例，或在 IDE 中执行 `src/test/java` 下所有测试类；
- 3) 测试报告：集成 `surefire-report` 插件，执行 `mvn surefire-report:report` 生成 HTML 格式测试报告，包含用例执行数、通过率、失败原因等。

5. 结果验证标准

- 1) 权限逻辑正确性：商家用户仅能操作自己的商家，管理员可操作所有商家，异常场景（如越权操作）需抛出指定 `APIException`（含正确错误码）
- 2) 状态流转正确性：商家申请新增时，管理员创建直接激活（`status=1`），普通商家创建需待审核（`status=0`）；逻辑删除后 `is_deleted=1`，且无法被正常查询
- 3) 排序逻辑正确性：`getBusinessesBySearch` 按 “评分 / 销量” 排序时，结果需与预期顺序一致（如销量降序时高销量商家在前）
- 4) 依赖调用准确性：通过 `Mockito.verify` 确认 `BusinessMapper` 的 `patchBusiness/insertBusiness` 等方法仅在权限通过时执行，越权场景下不执行。

6. 缺陷处理

若出现“商家越权更新成功”、“状态流转错误”等问题，优先排查 BusinessService 中的权限判断逻辑(如 isIdPresent 方法的空值处理)、SecurityContext 初始化是否正确；若搜索排序异常，检查 Comparator 排序规则是否与业务需求一致。

5.4 前后端联合测试

前后端联合测试聚焦“前端操作→接口调用→后端处理→前端渲染”全链路验证，确保前后端数据交互一致性、业务逻辑完整性。

5.4.1 测试范围与核心场景

表 5-4 核心场景

测试场景分类	具体场景描述	涉及前端模块	涉及后端核心接口	测试重点
实时订单交互	1. 顾客下单→商家端待接单列表实时出现订单 2. 商家接单→顾客端订单状态同步更新 3. 商家拒单→顾客端订单状态同步更新 4. 顾客取消订单→商家端订单状态同步更新	顾客下单页、商家订单页、顾客订单页	/api/orders/status 、 /api/orders/submit 、 /api/orders/detail	WebSocket 实时性、状态同步准确性
商家管理闭环	1. 商家申请新增店铺→管理员审核→商家端店铺列表更新 2. 商家更新店铺信息→前端实时渲染 3. 商家删除店铺→前端列表移除	管理员审核页、商家店铺管理页	/api/permission/apply-shop 、 /api/permission/audit	业务流程完整性、数据一致性
搜索与筛选	1. 顾客按关键词搜索商家→前端展示匹配结果 2. 顾客按“评分 / 销量”排序→前端按规则展示 3. 顾客筛选“免配送费、起送费在某个区间”的商家→结果精准过滤	顾客首页	/api/businesses/search	搜索匹配准确性、排序逻辑正确性、筛选有效性
其他	1. 顾客对地址的增删改查→前端正常渲染 2. 商家对商品的增删改查→前端正常渲染 3. 顾客点赞、收藏商铺→前端正确显示商铺点赞数与收藏数，正确显示收藏列表	地址管理页、顾客端我的页、商铺详情页、商家端我的页面	/api/addresses 、 /api/foods/status	正确完成业务流程

5.4.2 关键问题排查与验证方法

在联合测试中，针对“前后端数据不一致”“实时推送延迟”等常见问题，采用以下排查方法：

(1) 数据交互一致性验证

前端操作后，通过浏览器开发者工具的“Network”面板查看接口请求参数，同时查看接口是否正确返回数据。

(2) WebSocket 实时性验证

打开浏览器开发者工具的“Network→WS”面板，找到 `ws://test-api.xxx.com/merchant-ws` 连接，点击“Frames”查看推送消息；顾客下单后，观察“Frames”面板是否在 1 秒内收到 `type=order_pending` 的消息，消息是否含 `orderId`、`merchantId` 等关键信息；若未收到推送，先检查后端 `WebSocketServer` 日志（是否成功找到商家连接），再检查前端 `WebSocket` 初始化代码（是否正确携带 `merchantId` 参数）。

(3) 权限拦截一致性验证

使用普通用户账号，手动在地址栏输入管理端页面 URL。

(4) 预期

后端接口若被调用，应返回 `code=403`（权限拒绝），避免前端拦截失效导致越权访问。

第六章 部署文档

6.1 开发环境

(1)后端部分

基于 SpringBoot, Maven, Mybatis

开发环境:

- JDK 17
- SpringBoot 3.4.6
- Mybatis 3.0.4

(2)中间件

- Nginx 作为反向代理服务器

(3)网络

- 具备服务器公网 IP 地址, 以便外部访问

(4)数据库

基于 MySQL

配置信息:

- 主机: 110.42.60.144
- 端口: 3306
- 数据库名称: elm_v2
- 用户名: elm_v2
- 密码: zhijiage123

表结构如图6-1。

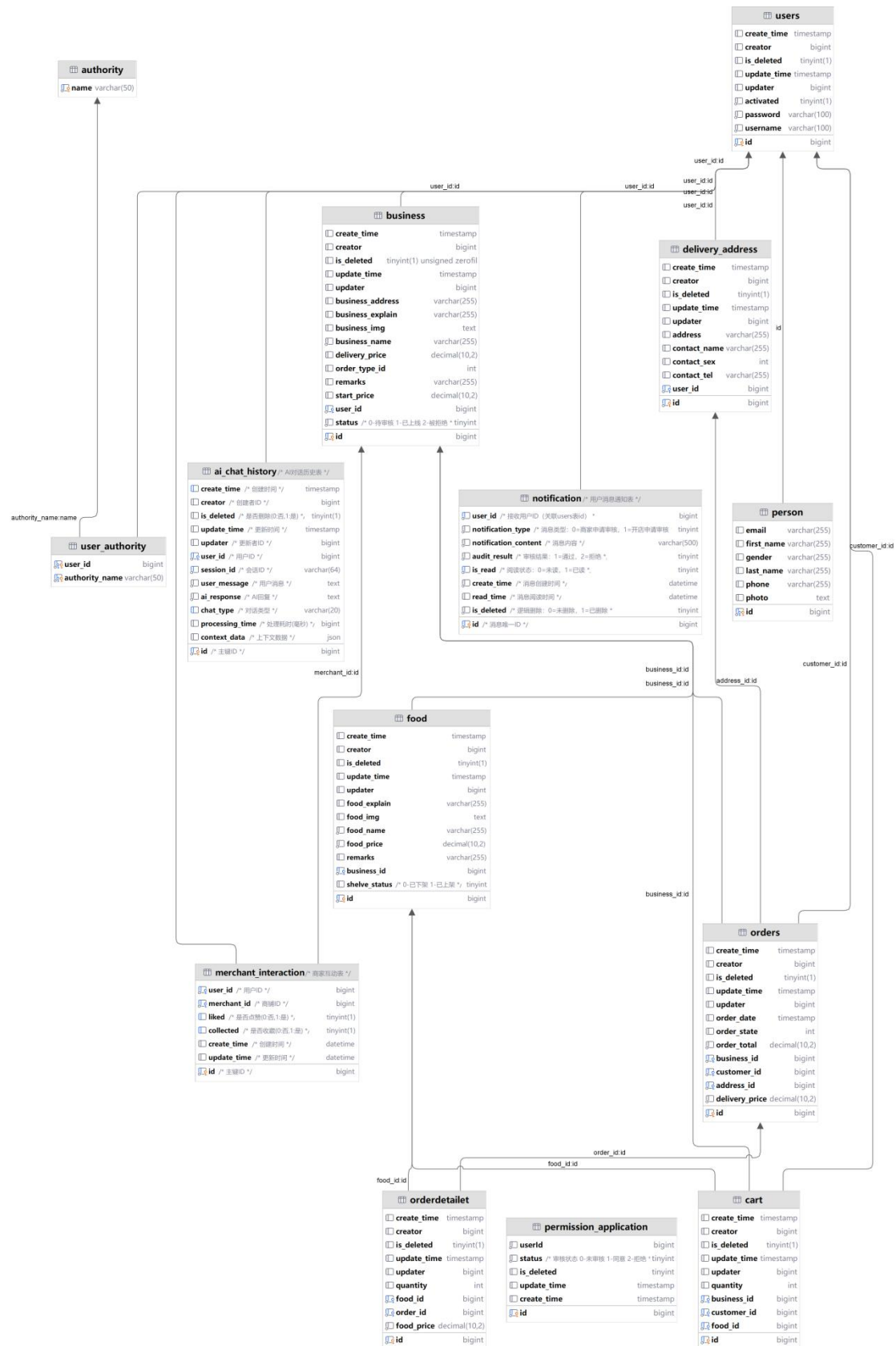


图 6-1 数据库表结构概览

6.2 运维需求

(1) 持续集成/持续部署 (CI/CD)

- 使用宝塔面板实现构建、测试和部署流程。

(2) 日志管理

- 在宝塔面板，可以轻松收集和分析系统日志，快速定位问题。

6.3 部署的步骤和流程

(1) 服务器配置

1. 购买服务器
2. 安装 Nginx 和 MySQL
3. 安装 Java 和 Spring Boot

(2) 前端部署

1. 将前端代码部署到服务器上
2. 配置 Nginx，将前端请求重定向到服务器

(3) 后端部署

1. 将后端代码部署到服务器上
2. 配置 Nginx，将后端请求重定向到服务器

(4) 数据库部署

1. 将数据库部署到服务器上

(5) 其他配置

1. 安装宝塔面板，方便操作
2. 配置 Nginx，将前端和后端请求重定向到服务器
3. 配置日志管理，快速定位问题

6.4 部署展示

可以直接在浏览器访问：110.42.60.144:8101，即可出现首页页面。

手机端展示（部分页面）：

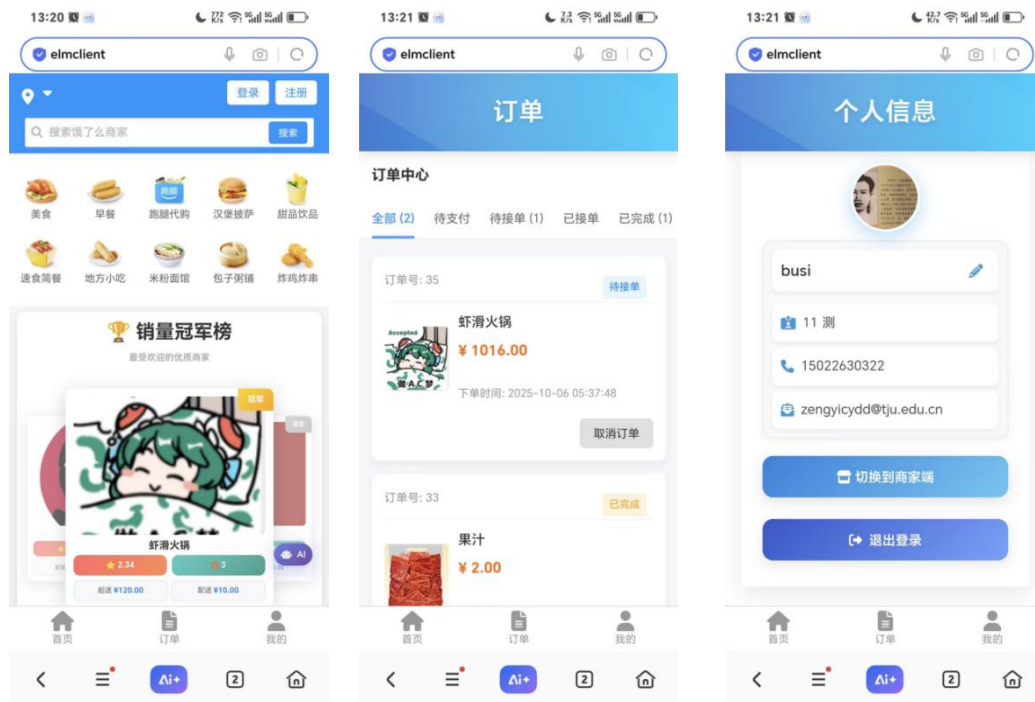


图 6-2

第七章 项目特色说明文档

7.1 文档目的与术语说明

7.1.1 文档目的

本文档旨在系统阐述项目在原有基础上实现的核心功能模块。项目参照“饿了么”的真实业务流程与产品逻辑，进行了功能增强与特色开发。下文将详细说明各用户角色的功能实现，并重点突出我们的创新点与项目特色。

7.1.2 术语定义

为便于理解，特对关键术语进行统一说明：

- 用户：泛指系统的所有使用者，包括顾客用户和商家用户
- 商家：特指拥有店铺管理、商品管理等后台操作权限的用户
- 商铺：由商家创建的线上店铺。一个商家用户可创建并管理多个商铺
- 管理员：拥有最高权限的系统管理者，负责审核、管理用户与商铺

7.2 系统功能详述

本项目构建了覆盖顾客端、商家端和管理端的完整业务体系，以下是各端核心功能概述。

7.2.1 顾客端功能

1. 首页与发现

- 1) 智能定位：集成高德地图API，实现真实地址选择
- 2) 个性化推荐：以轮播图形式展示销量前三的热门商家，支持手动翻页与快捷跳转
- 3) 高效检索：提供商铺名、商铺简介及商铺地址的模糊搜索、按食品分类（如早餐、甜品）筛选、综合/销量排序以及配送费、起送价等多维度筛选功能
- 4) AI智能助手：接入DeepSeek API，提供悬浮式AI客服（小饿），支持实时问答与对话历史查询

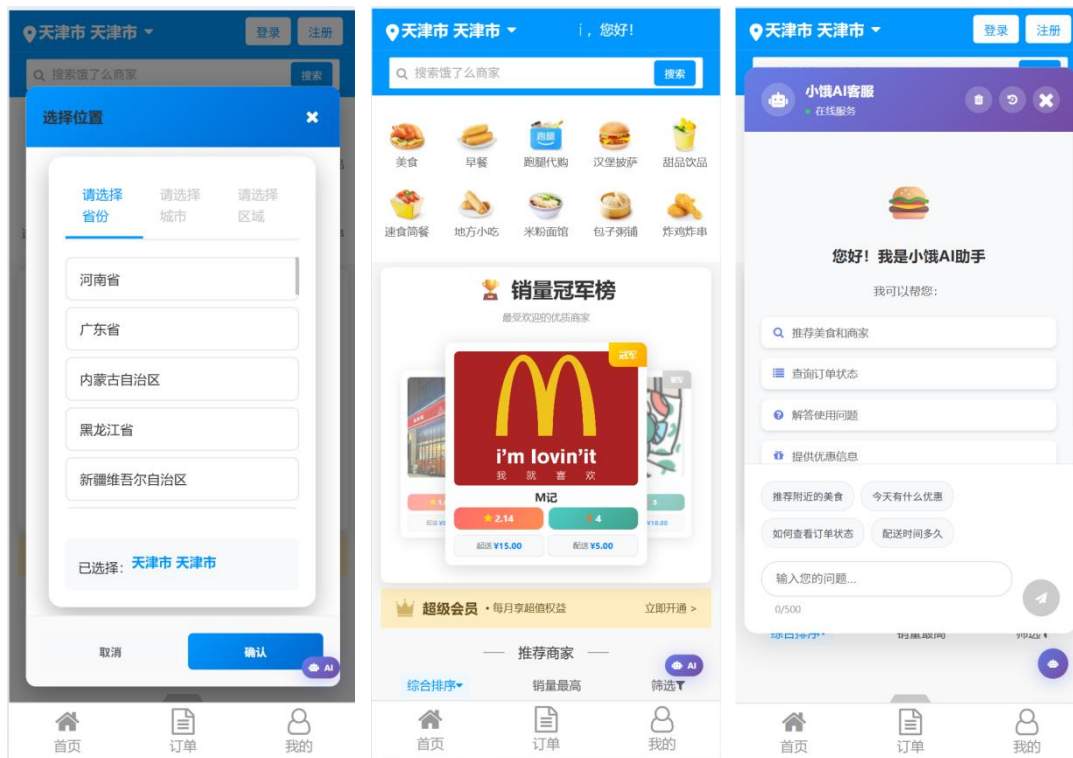


图7-1

2. 下单与购物车

- 1) 商家主页：展示商家简介，支持用户点赞、收藏店铺
- 2) 智能购物车：具备完整的增删改功能，并实时校验起送价，未达标时支付按钮置灰
- 3) 灵活配送：支付前可新增、编辑或选择配送地址
- 4) 订单确认：支付完成后清晰展示订单详情

3. 订单中心

- 1) 状态管理：通过状态栏（全部、待支付、待接单、已接单、已完成、已取消）分类查看订单
- 2) 订单操作：商家接单前，顾客可取消订单；接单后，可确认完成订单
- 3) 详情页面：点击订单可查看包含商品明细、费用、配送跟踪等完整信息
- 4) 数据持久化：商铺或商品信息的后续变更不会影响历史订单数据的准确性

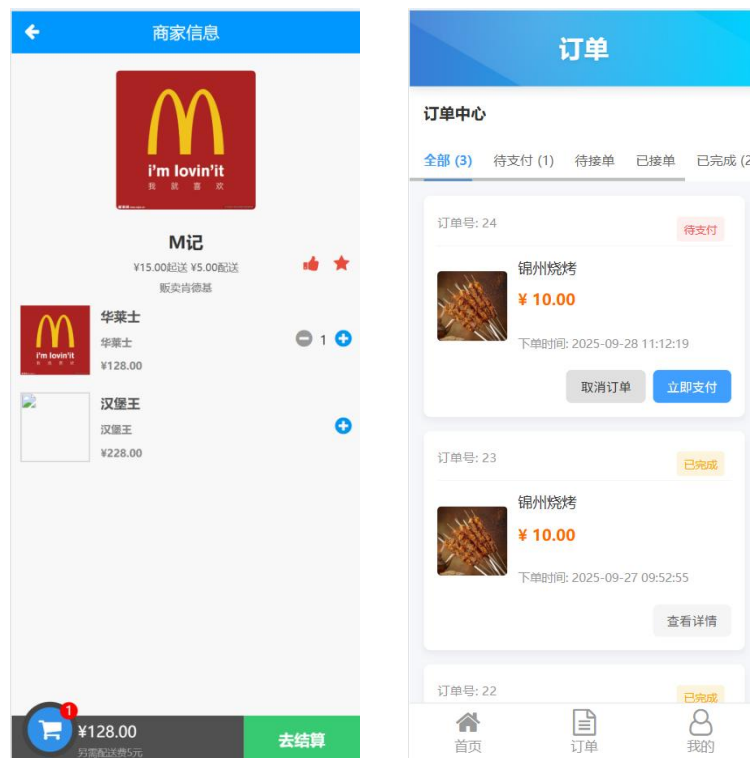


图7-2

4. 个人中心

- 1) 角色切换：顾客可提交商家身份申请，审核通过后可在顾客端与商家端间一键切换
- 2) 信息管理：管理收货地址、编辑个人资料（头像、昵称）、查看收藏夹与消息通知

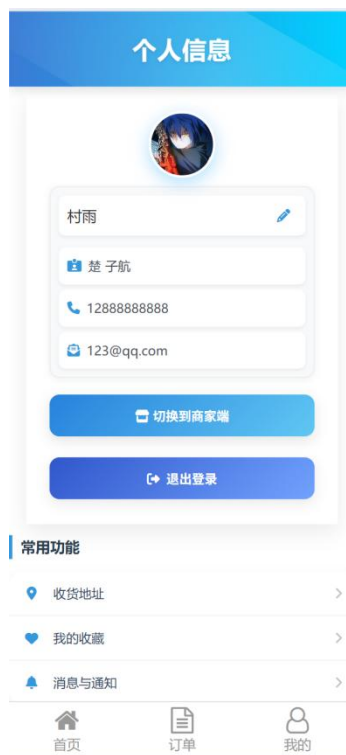


图7-3

7.2.2 商家端功能

1. 商铺管理

- 1) 状态化管：按“审核中/已上线/未通过”状态分类管理名下店铺，可进行信息编辑或删除
- 2) 开店申请：可向管理员提交新店开设申请
- 3) 商品管理：对每个店铺内的商品进行上架、下架、编辑和删除操作

2. 订单管理

- 1) 多维查看：可按店铺或订单状态（如待接单、已完成）筛选查看订单
- 2) 订单处理：可浏览订单详情，并进行接单、拒单、完成等操作

3. 数据看板

- 1) 店铺数据：分店铺查看点赞、收藏数量及综合评分情况
- 2) 个人信息：查看及管理商家账户的基本信息

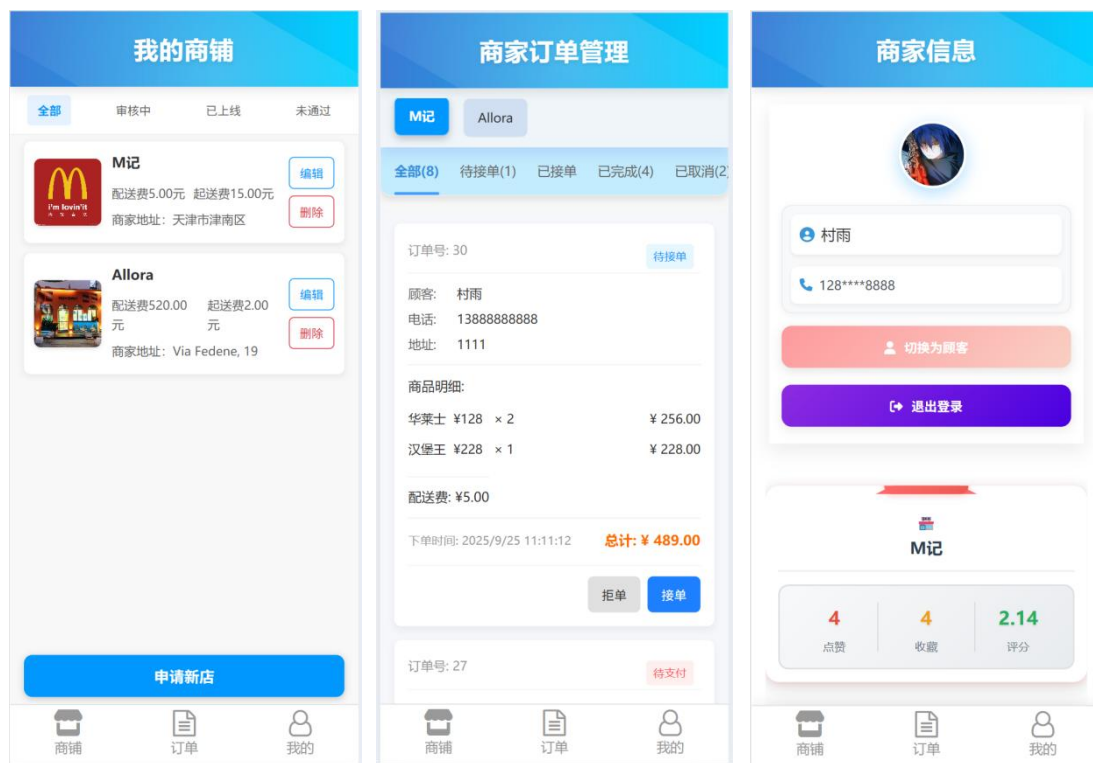


图7-4

7.2.3 管理端功能

1. 审核与概览

1) 统一审核：

- a) 用户审核：审核用户提交的商家资质申请，可查看其个人信息并决定是否批准。
- b) 商铺审核：审核商家提交的新店申请，可查看店铺详情并决定是否上线。
- c) 数据看板：总览平台核心数据，如总用户数、总店铺数、总营业额等。

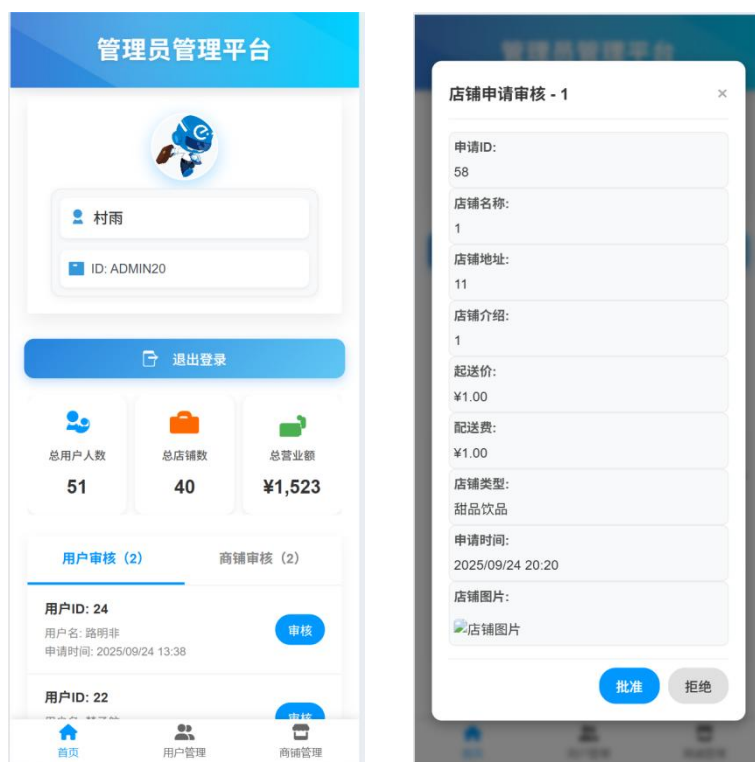


图7-5

2. 用户管理

- 1) 精准搜索：支持通过手机号、用户名、邮箱等条件搜索用户。
- 2) 状态控制：可按状态（全部/已启用/已禁用）管理用户，并执行启用或禁用操作。

3. 商铺管理

- 1) 集中管理：可按商家维度管理其对应的所有商铺。
- 2) 高级操作：管理员拥有直接新增、编辑或删除任何店铺的权限，此操作无需经过审核流程。

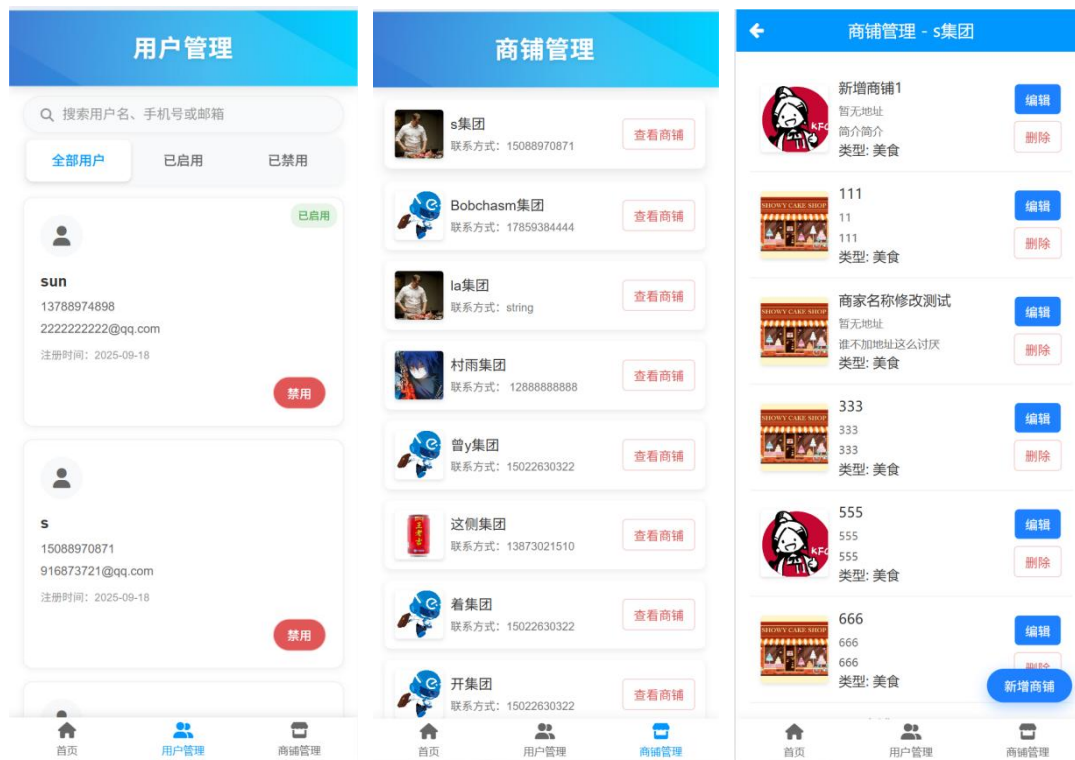


图7-6

7.3 项目特色与创新点

(1) 三端一体化的完整闭环：构建了顾客、商家、管理员三个角色分离且权责清晰的完整业务系统

(2) 智能身份切换：登录时无需选择身份，系统根据权限自动跳转；商家用户可在通过审核后无缝切换身份

(3) 增强型用户体验：

1. AI集成：引入AI大模型提供智能客服，提升交互体验

2. 数据持久化：确保历史订单数据不受后续商家信息修改的影响，保证交易记录的准确性

3. 实时通知：利用 WebSocket 技术，实现审核状态等消息的实时推送，无需手动刷新页面

4. 精细化的运营管理：为管理员提供了强大的用户管理、商铺直接管理及数据看板功能，赋能平台运营

第八章 问题与解决&结果反思

1. git 合并代码时出现冲突，解决冲突不当，导致部分代码修改丢失

解决：合并时成员在线下共同讨论每一个冲突处应该保留哪一方代码

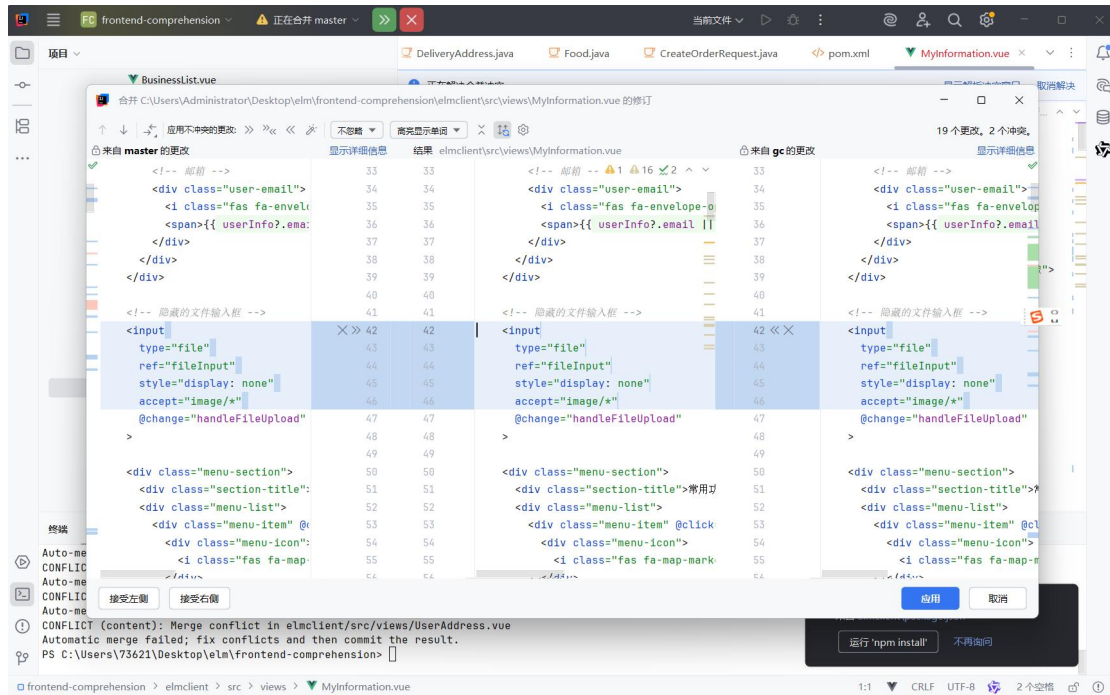


图 8-1 git分支合并

2. 代码请求中存在异步操作（接口请求），操作完成前依赖的数据还没准备好，导致后续读取属性时数据为 undefined

解决：通过 async/await，保证在数据准备好之后再执行读取属性的操作

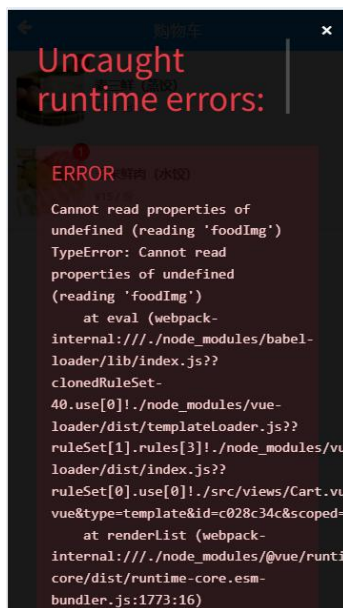


图 8-2 异步请求

3. WebSocket 失效，浏览器开发者控制台一直报错无法连接

解决：通过浏览器开发者控制台报错码可知是由于 jwt 拦截器拦截了 WebSocket 握手请求，在后端放行了/ws/** 后问题解决

4. 图片加载失败处理，一直请求某图片的链接

原因：自动将 .. 转为 localhost:8080

解决：路径解析问题，将../assets/default-image.png中的..改为@，
require('@assets/business-default.png')增加图片加载失败的处理：

```

```

图 8-3 图片加载

5. 用户对商家进行点赞收藏后图标变红，但重新加载进入页面后，点赞和收藏的颜色变回了未操作的颜色

解决：取消请求头缓存设置

```
const response = await request.get('/api/merchant/interaction/status', {  
  params: { userId: userId, merchantId: businessId.value },  
  // headers: { 'Cache-Control': 'no-cache' }  
});
```

图 8-4 请求头缓存

6. 只要进入首页就会跳转到登录界面，用户没法注册

解决：通过浏览器开发者控制台的接口调用记录，发现是首页调用获取销量前三商铺、获取商铺列表和ai智能体相关接口因为未带 token 被响应拦截器拦截，导致跳转到登录页，在后端放行这些接口后解决该问题

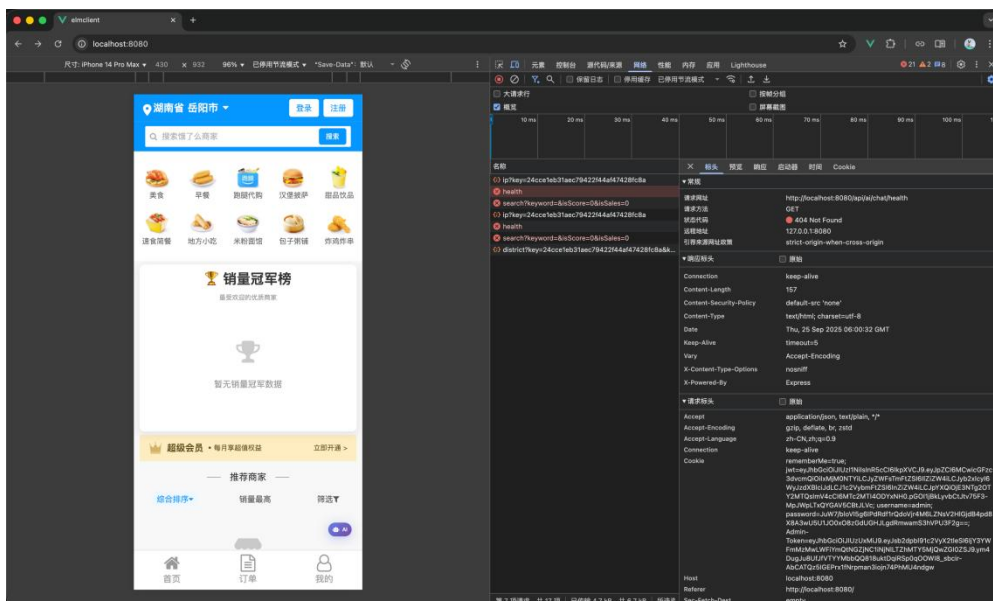


图 8-5 登录跳转

7.后端项目启动时，一直报错：找不到注解路径

解决：在设置中勾选上“从项目类路径获处理器”

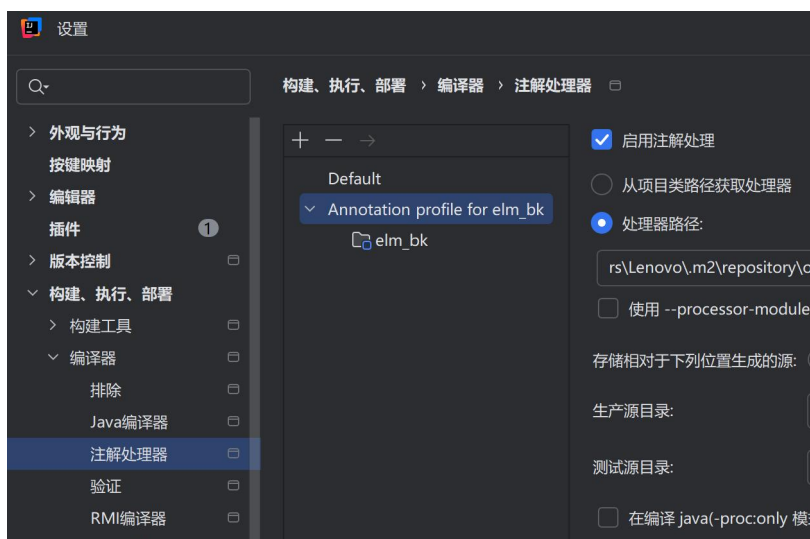


图 8-6 找不到路径

8.页面顶部或底部显示不全，有些页面在下拉的时候，由于底部有遮挡，没有办法下拉到最下边，还有一些页面顶部遮挡

解决：底部/顶部适当留白，增加外层容器 padding-bottom/padding-top 的值，并且有些页面添加了移动端响应样式 @media，需要同时将此处的外层容器的padding 值调到合适的大小

9.前端界面的样式统一

由于前后端分离且分工进行前后端开发，会遇到样式不统一的问题。

解决：采用统一的样式规范（例如background: #0097ff，边缘圆角采用：80px）+记录 and 讨论并即使改正

10.接口未完善时，前端的数据处理问题

解决：使用了axios 相应拦截器，在控制栏反馈了模拟接口的响应

11.项目部署时，后端接口可正常通过服务器ip访问，但访问前端页面时出现不支持

解决：将前端的配置文档配置的端口号改为 8101 并加上 API 代理配置

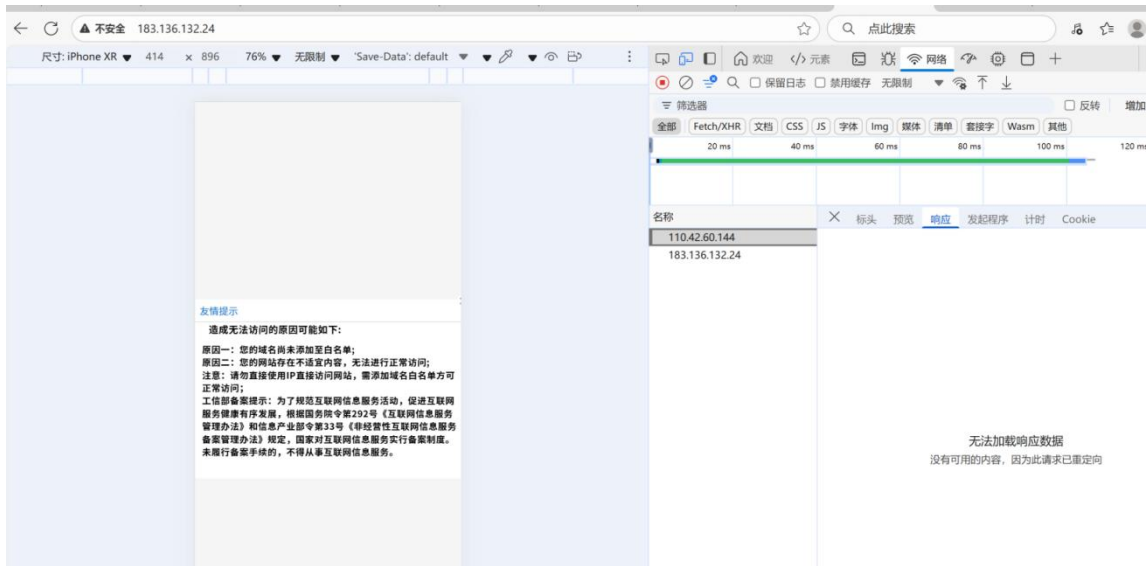


图 8-7 前端部署访问不支持

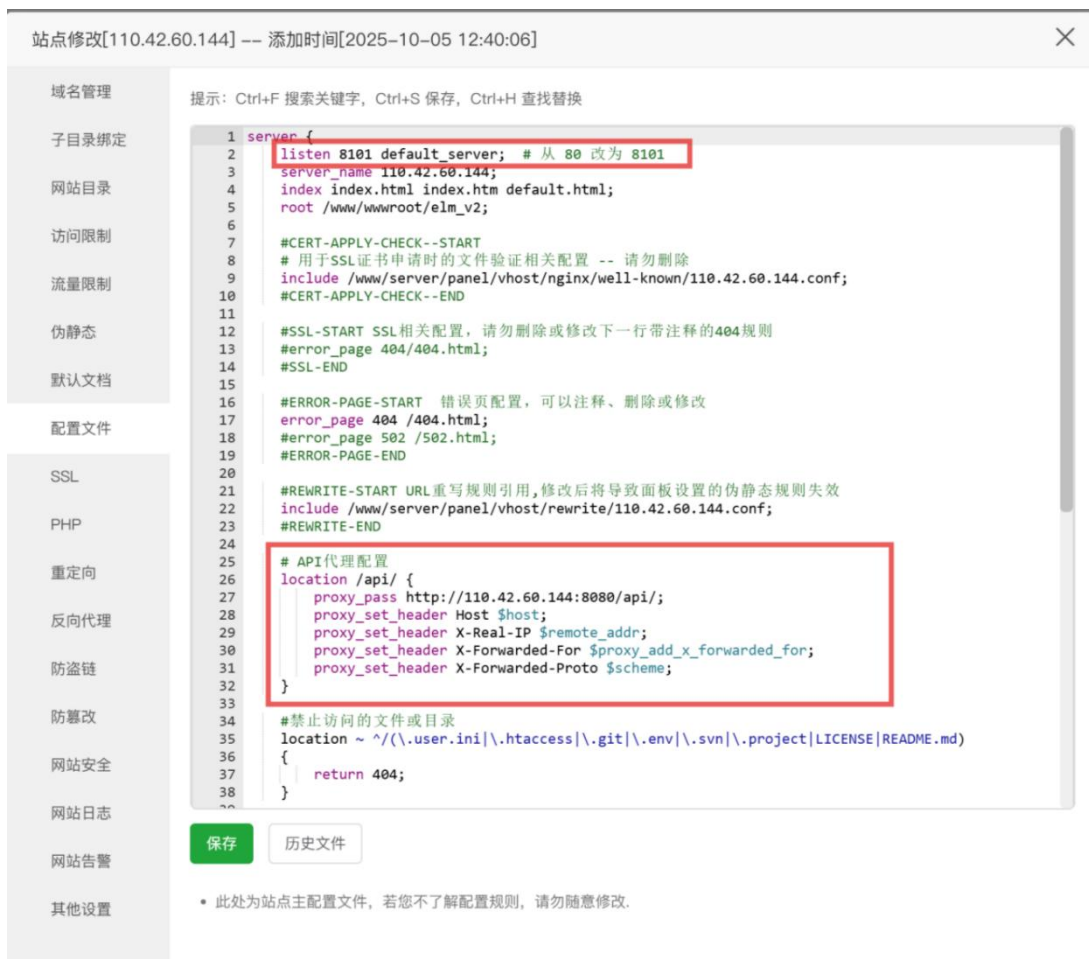


图 8-8 前端代理配置

第九章 实践总结与展望

9.1 实践总结

在本次综合实践中，小组成员协力合作，基本上完成了一次典型的前后端分离项目开发从确定需求与设计、开发到测试验收的全流程。

经过此次实践，初步了解了一个软件开发项目的推进与多人协作流程；掌握了经典前后端分离项目 `Springboot`、`vue` 等框架的基本知识和实践能力；进一步学习重要的 `git` 版本控制在多人协作中的使用¹，以及在分支管理时遇到的冲突解决；最重要的是，领悟到软件开发前需求分析的重要性，以及在开发过程中细节确定的重要性，合作者需要进行有效的及时的沟通。

对于开发过程，实际项目里需求是不断变化的。要注意代码的解耦，以及全局设置的提炼整合，为后续的修改和优化留出空间，使得整体改动尽可能小，降低维护难度。同时项目需要进行多轮的测试，一次写好无 `bug` 的程序是不可能的，要贯彻好 `KISS` 原则，不断优化项目，提升用户体验。

9.2 未来展望

在本次实践中，主要将项目的业务功能完善，使得用户可以体验比较完整的功能链。但没有考虑实际复杂情况下可能出现的问题，下个阶段，我们会考虑更为大量用户带来的并发问题，进行压力测试，考虑使用一些如 `Redis`，`MQ` 等经典中间件，提高 `QPS` 等并发指标；同时，进一步完善系统功能，使系统更接近于实际应用，可以考虑微服务或其他系统优化，并进一步提高安全性，如限制多端登录等。

小组将在未来的实践中不断提升能力，逐步使“饿了么”项目贴近与现实应用。

¹ 仓库链接: <https://gitee.com/dai-mingjing/frontend-comprehension.git>